



University
of Basel

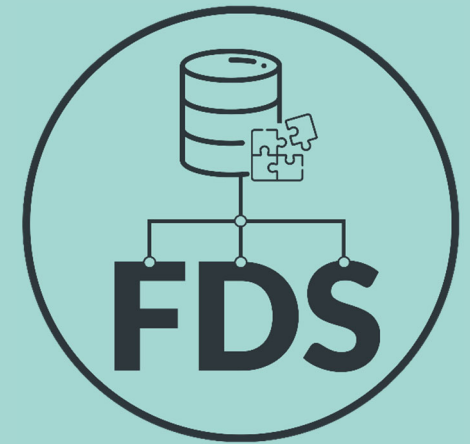
Chapter 4: Data

4.2: Data Consistency

Foundations of Distributed Systems

Fall Semester 2025

Prof. Dr. Heiko Schuldt



Overview of Chapter 4

4.1	Self-Organization (P2P)
4.2	Data Consistency (Distributed Transactions)
4.2.1	Foundations of Transaction Management (partly recap from cs246 Database Systems)
4.2.2	Atomicity of Distributed Transactions
4.2.3	Serializability of Distributed Transactions
4.3	Data Replication

Objectives of Chapter 4.2 – Data Consistency / Distributed Transactions

- Concurrency control and recovery are well known and established concepts in centralized database systems to guarantee that applications (transactions) can be executed correctly, even in case of concurrency and failures.
- However, proper failure handling and dealing with parallel accesses are also essential in distributed databases, when transactions span two or several databases.

You will ...

- review the main transaction concepts in centralized databases and how they can be implemented,
- learn how atomic transaction execution can be guaranteed even for distributed transactions,
- generalize the concept of concurrency control to distributed transactions.

Overview of Chapter 4.2 – Data Consistency

4.1	Self-Organization (P2P)
4.2	Data Consistency (Distributed Transactions)
4.2.1	Foundations of Transaction Management (partly recap from cs246 Database Systems)
4.2.2	Atomicity of Distributed Transactions
4.2.3	Serializability of Distributed Transactions
4.3	Data Replication



What is a Transaction?

- A transaction is a **set of logically corresponding operations**
 - Traditional database transactions consider **read and/or write operations of database pages** in a single database system
 - Distributed transactions jointly provide access to different database systems which has to be coordinated
- Transactions are based on a generic concept to encapsulate operations and to provide dedicated guarantees for sequences/sets of operations
 - Execution guarantees: **ACID**
 - Guarantees are provided by the runtime system, transparently for users/programmers
 - Developers/users simply have to specify **transaction boundaries** (BOT = Begin-of-Transaction and EOT = End-of-Transaction). They don't have to worry about how the ACID guarantees will be provided.



Transaction Properties: ACID

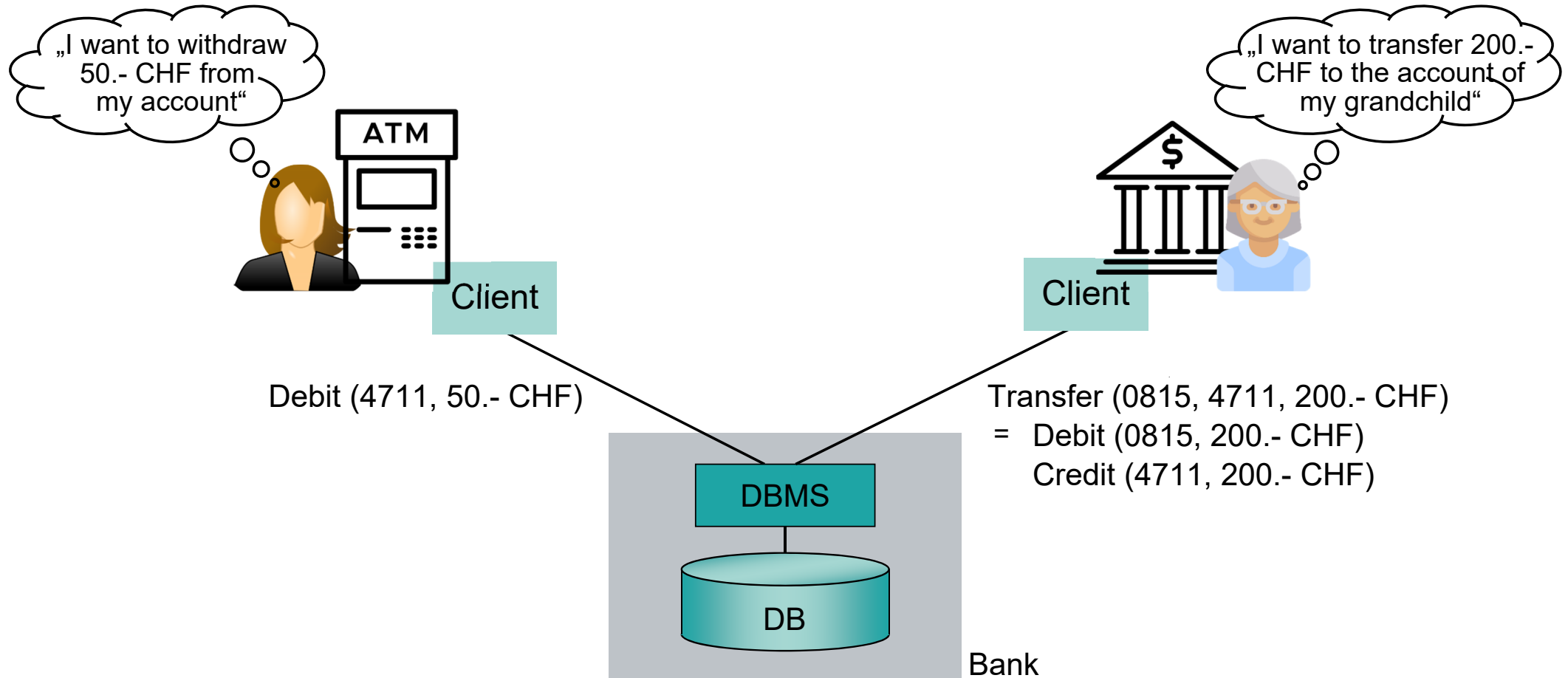
- **Atomicity**
 - A transaction is either executed completely (“commit”) or does not leave any effects (appears as if it had never been started), i.e., it is aborted (“rollback”).
Transaction atomicity thus corresponds to an “All-or-Nothing” semantics.
- **Consistency**
 - A transaction transfers one consistent database state into another consistent database state
- **Isolation**
 - Transactions, even when being executed in parallel, appear as if they were executed in isolation (i.e., sequentially)
- **Durability**
 - After a transaction has committed, its effects are persistent (they even survive system crashes)



Enforcing ACID Properties

- Transactions can be considered as **contract between the application program and the runtime system** of the database
 - ACID properties are the contents of this contract
- The most important tasks of a database system regarding transaction management include
 - The correct synchronization of concurrent transactions
(*Concurrency Control* → *isolation*)
 - The correct treatment of system crashes and application failures
(*Recovery* → *atomicity, persistence*)
- For both areas, first a definition of correctness criteria is needed
 - Concurrency Control: serializability theory
 - Recovery
- On the basis of these criteria, protocols for enforcing them are required
 - Dedicated components exist that implement these protocols

Bank Transfer: Scenario



Transaction Management: Atomicity



⋮

↓ t

AcctID	Balance
0815	2770.-
4711	120.-

2570.-

AcctID	Balance
0815	2570.-
4711	120.-



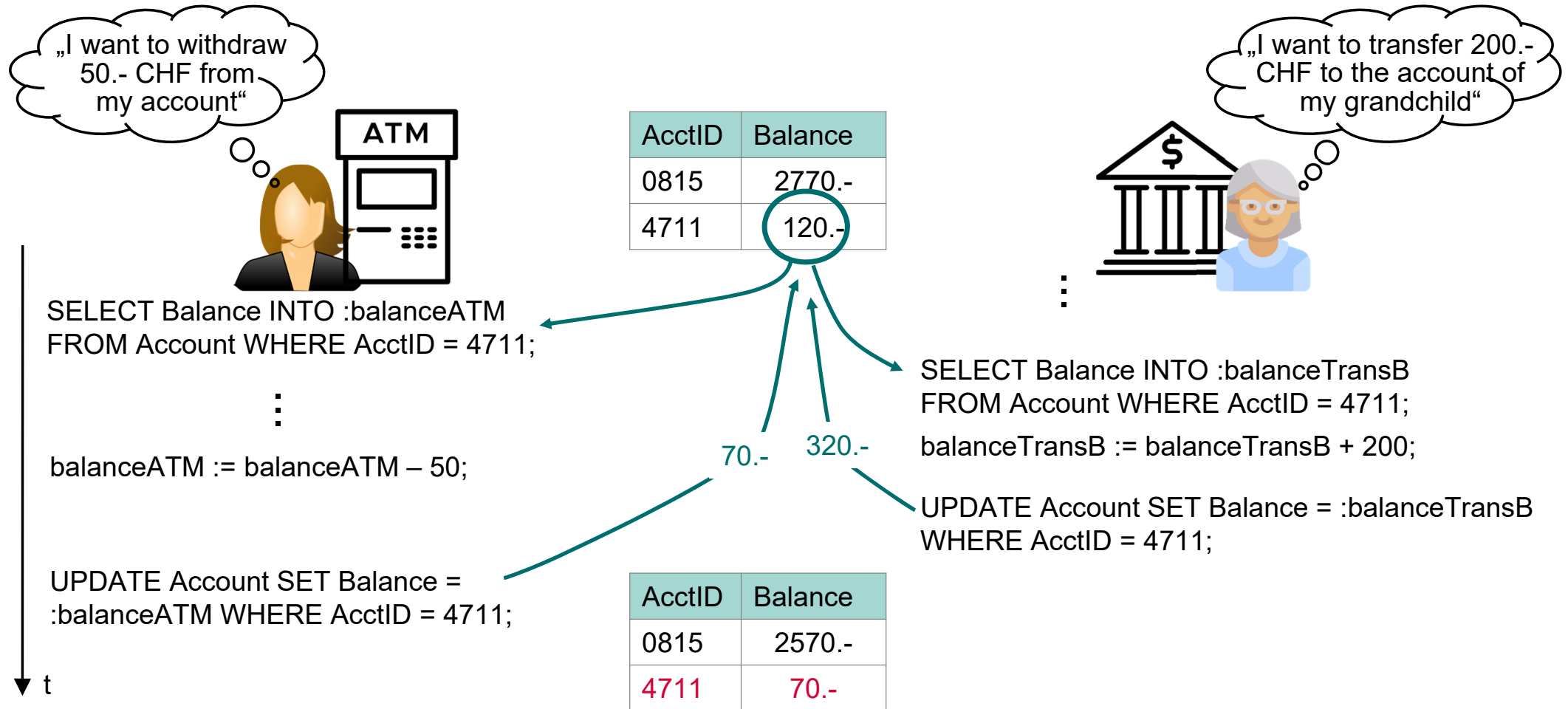
```
SELECT Balance INTO :balanceTransA
FROM   Account WHERE AcctID = 0815;
balanceTransA := balanceTransA - 200;

UPDATE Account SET Balance = :balanceTransA
WHERE  AcctID = 0815;

SELECT Balance INTO :balanceTransB
FROM   Account WHERE AcctID = 4711;
```

*** Program Crash ***

Transaction Management: Isolation



Read/Write Model



- The main concepts of concurrency control and recovery will be discussed on the basis of the read/write model (**r/w model**) that focuses on the actual I/O operations the database system performs
 - Basic notions
 - **Database (DB)** Set of objects: $DB = \{o_1, o_2, \dots\}$ (for instance: database pages)
 - **Operation (OP)** Set of operations: $OP = \{\text{read}, \text{write}\}$
 - **Action** Action $a \in (OP \times DB)$, i.e., either $\text{read}(o_i)$ or $\text{write}(o_i)$.
 - $\text{read}(o_i)$ returns database object o_i without having any side-effects
 - $\text{write}(o_i)$ changes database object o_i
- Abbreviations:
- $r(o_i)$ for $\text{read}(o_i)$
 - $w(o_i)$ for $\text{write}(o_i)$
- Hence, SQL operations need to be mapped to operations in the R/W model

Transaction



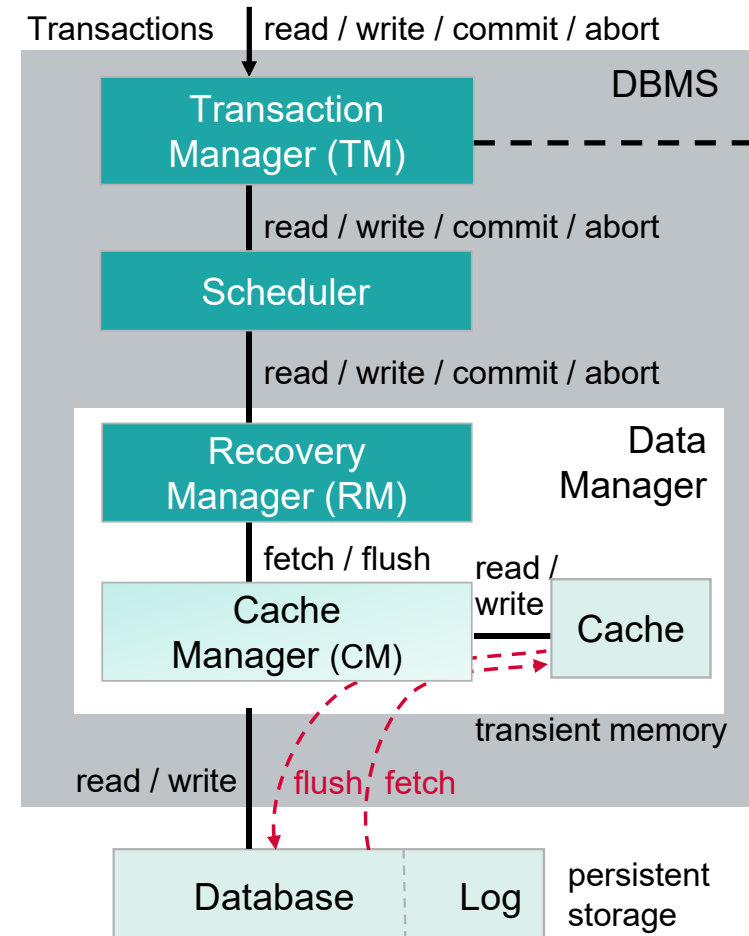
Transaction (T) A transaction T is a tuple with $T = (ACT, <)$ where ACT is a set of actions on which a (partial) order $<$ is defined, i.e.,
 $ACT = \{a_1, a_2, \dots, a_k\} \cup \text{term}$
 $a_1, a_2, \dots, a_k$ are **actions**
 $\text{term} \in \{C, A\}$ is a **terminating action**
 $< \subseteq (ACT \times ACT)$ is the **precedence relation**

- Each transaction T is terminated either by **C (commit)** or **A (abort)**. C or A, respectively, are ordered after all other actions of a transaction with regard to $<$, i.e., $a_i < \text{term}$ for all $1 \leq i \leq k$.
- The **precedence relation** $<$ specifies the execution order of all actions of a transaction. It can be
 - $<$ partial: allows for parallelism within a transaction (**intra-transaction parallelism**)
 - $<$ total: **sequential execution** $T = (a_1 < a_2 < \dots < a_k < C)$

DB System Model and Components



- The Transaction Manager (TM) forwards incoming operations to the scheduler. In case of distributed DBMS, this includes the communication as well as the coordination of distributed transactions with remote TMs.
- The **Scheduler** decides whether operations can be executed, have to be deferred, or even rejected.
- The **Recovery Manager (RM)** protects the DB from system failures (e.g., when the transient storage is lost), it guarantees that effects of successfully completed transactions are stored persistently, and that aborted transactions do not leave any side effects
- The Cache Manager (CM) guarantees an efficient access to data



Serializability Theory



- Serializability addresses the correctness criteria for the correct execution of concurrent transactions.
 - **Given:** $\tau = \{ T^1, T^2, \dots, T^n \}$ a set of n concurrent transactions that shall be executed correctly.
 - **Basic Assumption A1:** Each transaction T^i is correct, when being executed in isolation.
 - **Basic Assumption A2:** Each serial (sequential) execution of all n transactions (in any order) is correct. Different transactions are supposed to originate from different independent users/applications; for the latter, any sequential execution is acceptable.
- We need criteria that allow to check whether a given parallel execution is equivalent to such a serial execution!

Schedule ...



- The parallel execution of all transactions in τ is reflected by a so-called **schedule**:

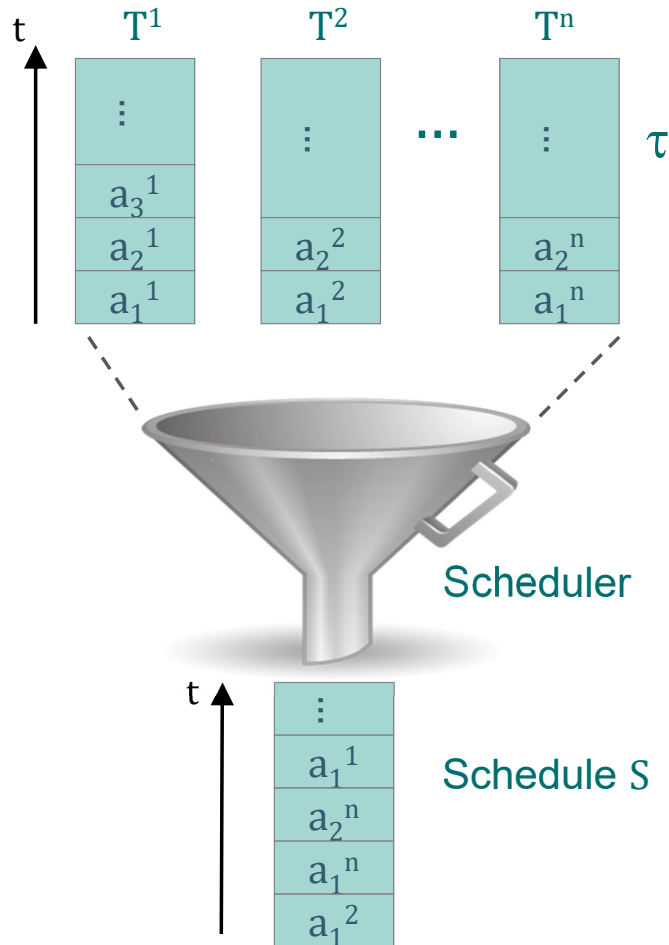
Complete schedule S A complete schedule S is a triple $S = (\tau, \mathbb{A}, <)$. S contains the execution order of all actions of all transactions from τ in which all precedence relations are kept.

with:

$\tau = \{ T^1, T^2, \dots, T^n \}$	set of transactions
$\mathbb{A} = \bigcup ACT^i$	all actions of all transactions in τ (*)
$< \subseteq (\mathbb{A} \times \mathbb{A})$	partial order, with $<^i \subseteq <$ for all $T^i \in \tau$ i.e., the execution order within the schedule respects the internal precedence order of each transaction

(*) A **complete schedule** contains either exactly a “C” or an “A” for each transaction.

... Schedule ...



Schedule: A schedule is a **prefix** of a complete schedule

- A schedule S contains
 - **Completed** (C in S)
 - **Aborted** (A in S)
 - **Active** (neither C nor A in S) transactions
- A complete schedule does not contain any active transactions
- The order $<$ of a schedule is:
 - $<$ **total**: sequential execution
 - $<$ **partial**: parallel execution e.g., in distributed databases

... Schedule



Serial schedule: In a serial schedule, $<$ is total and for any $i, k \in \{1, \dots, n\}$: all actions of T^i are executed before all actions of T^k

$$S_{\text{ser}} = \underbrace{a_1^2 a_2^2 \dots C^2}_{T^2} \underbrace{a_1^n a_2^n \dots C^n}_{T^n} \underbrace{a_1^1 a_2^1 \dots C^1}_{T^1}$$

- Two serial schedules are considered correct usually return different database states.

Committed Projection: the committed projection $C(S)$ of a schedules S is defined by removing all actions of transactions that have not committed yet, i.e., τ is restricted to $\bigcup T^i: C^i \in S$

- $C(S)$ neither contains active nor aborted transactions

Semantics of Actions



- The semantics of actions is defined as follows:

read: $r^i(x)$ reads object x from $w^k(x)$, where $w^k(x)$ is the last write action in S before $r^i(x)$.

But: $i \neq k$ and T^k is not aborted.

write: $w^i(x)$ provides a new version of the object that replaces the old one.

The write depends on all preceding readers of T^i before $w^i(x)$.

→ The result of a write action thus depends (in an application-specific way) on all preceding read actions of the same transaction.

Reads-From Relation



Reads-From Dependency: Let S be a closed schedule. $r^i(x)$ reads-from $w^k(x)$, $i \neq k$, if:

- (a) $w^k(x) < r^i(x)$
- (b) $\neg \exists w^m(x)$ with $w^k(x) < w^m(x) < r^i(x)$
- (c) T^k is not aborted in S when $r^i(x)$ is executed, i.e., $\neg \exists A^k < r^i(x)$

Reads-From Relation (RF): $RF = \{ (T^k, x, T^i) \mid r^i(x) \text{ reads-from } w^k(x) \}$

Conflict Serializability



- Conflict serializability is based on the general observation that the order of **some actions in a schedule can be swapped without any effects** (both from the point of view of the actions/transactions involved and from the point of view of the corresponding objects)

- Example:

$$\begin{array}{ll} r^1(x) < r^2(x) & \sim r^2(x) < r^1(x) \\ w^1(x) < w^2(y) & \sim w^2(y) < w^1(x) \end{array}$$

- Conversely, there are **pairs of actions that cannot be swapped** (as such a swap would have consequences on these actions and the objects they have accessed). In such cases, a **dependency** exists.

- Example:

$$\begin{array}{ll} r^1(x) < w^2(x) & \not\sim w^2(x) < r^1(x) \\ w^1(x) < w^2(x) & \not\sim w^2(x) < w^1(x) \end{array}$$

Conflict Relation



- Actions that cannot be swapped are considered to be in conflict. This is reflected in the **conflict relation** con .
- **Conflict relation** con :
 $\text{con}(a, a') \Leftrightarrow$ (1) a and a' operate on the same DB object
(2) $a = \text{read}(x) \wedge a' = \text{write}(x)$ **r/w conflict**
 $\vee a = \text{write}(x) \wedge a' = \text{read}(x)$ **w/r conflict**
 $\vee a = \text{write}(x) \wedge a' = \text{write}(x)$ **w/w conflict**
- In the read/write model, two actions are in conflict if they access the same database object any of these two actions is a write.

Dependency Relation



- The **dependency relation** of a schedules S contains all conflict pairs of all transactions in S .

Dependency & dependency relation:

Let S be a schedule. Action a^k is dependent on action a^i in S , short: $a^i \rightarrow a^k \Leftrightarrow$

- | | | |
|-----|------------------------|--|
| (1) | $i \neq k$ | T^i and T^k are different transactions, |
| (2) | $a^i < a^k$ | a^i is before a^k in S , |
| (3) | $\text{con}(a^i, a^k)$ | a^i and a^k are in conflict, |
| (4) | C^i, C^k in S | both transactions are completed (with commit). |

- The **dependency relation** $\text{dep}(S)$ is the set of all dependent pairs in S : $\text{dep}(S) = \{(a^i, a^k) \in S \mid a^i \rightarrow a^k\}$
- Example:

$$S_1 = \langle \underbrace{w^1(a) \ r^2(a)} \ \underbrace{r^1(b) \ w^2(b)} \ C^1 \ C^2 \rangle$$

dependent: $w^1(a) \rightarrow r^2(a), \ r^1(b) \rightarrow w^2(b)$
 $\text{dep}(S_1) = \{(w^1(a), r^2(a)), (r^1(b), w^2(b))\}$

$$S'_1 = \langle \underbrace{w^1(a) \ r^1(b)} \ \underbrace{C^1 \ r^2(a)} \ w^2(b) \ C^2 \rangle$$

dependent: $w^1(a) \rightarrow r^2(a), \ r^1(b) \rightarrow w^2(b)$
 $\text{dep}(S'_1) = \{(w^1(a), r^2(a)), (r^1(b), w^2(b))\}$

Conflict Equivalence



- The dependency relations allows to capture the semantics of a schedule and thus to compare two schedules over the same set of transactions.

Conflict Equivalence:

Two schedules S and S' are **conflict equivalent**, if both schedules

- contain the same actions
- have an identical dependency relation, i.e., $\text{dep}(S) = \text{dep}(S')$

- In case information flows between two transactions T^i and T^k , this can only happen via conflict pairs of T^i and T^k , thus via shared database objects. Hence, the order of conflict pairs is essential for the direction of information flow.
- Example (cont'd):

$$S_1 = \langle w^1(a) \ r^2(a) \ r^1(b) \ w^2(b) \ C^1 \ C^2 \rangle$$
$$S'_1 = \langle w^1(a) \ r^1(b) \ C^1 \ r^2(a) \ w^2(b) \ C^2 \rangle$$

S_1 and S'_1 are conflict equivalent,
since $\text{dep}(S_1) = \text{dep}(S'_1)$



Conflict (Preserving) Serializability (CPSR)

Conflict (preserving) serializability (CPSR):

A schedule S is **conflict serializable**, if its committed projection $C(S)$ is conflict equivalent to a **serial schedule** S_{ser} . The class of all conflict serializable schedules is called **CPSR**.

- Conflict equivalent schedules can thus be produced by swapping actions that are not dependent.
- Observation: Since CPSR focuses on the committed projection of a schedule, the effect of aborted transactions is not considered.
- Example (cont'd):

$$S_1 = \langle w^1(a) \ r^2(a) \ r^1(b) \ w^2(b) \ C^1 \ C^2 \rangle$$

$$S'_1 = \langle w^1(a) \ r^1(b) \ C^1 \ r^2(a) \ w^2(b) \ C^2 \rangle$$

S_1 is CPSR, since S_1 and S'_1 are conflict equivalent and since S'_1 is serial.

Order Preserving Serializability (OPSR)



Order preserving serializability:

A schedule S is (transaction) **order preserving serializable**, if $S \in \text{CPSR}$ and if there is at least one equivalent serial schedule S' in which all completely ordered transactions in S have the same order in S' .

The class of order preserving serializable schedules is called **OPSR**.

$\text{OPSR} \subset \text{CPSR}$

Commit Order Preservation



- OPSR simply addresses the preservation of the order of transactions that are completely ordered in a schedule (i.e., that are non-overlapping), but not the order in which commits take place. The latter is also addressed by **commit order preserving serializability**.
- Basic idea:
 - Make sure that parallel transactions are executed in a serializable way and provide a means to specify the serialization order
 - This is done via the order in which commits take place

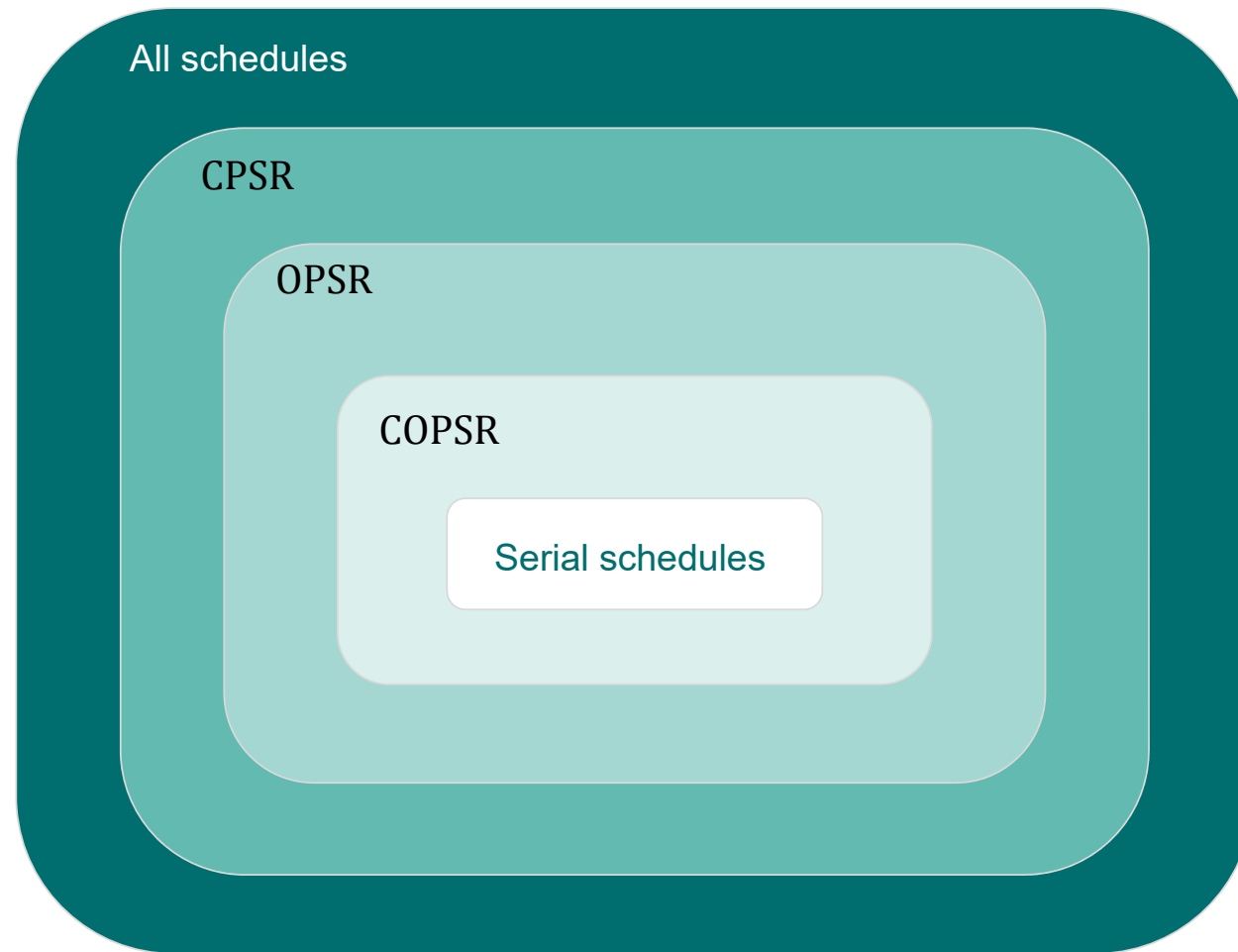
Commit Order Preserving Serializability:

A schedule S is **commit order preserving serializable** if the following holds for each pair T^i, T^k of committed transactions in S : if $(a^i, a^k) \in \text{dep}(S)$, then $C^i < C^k$.

The class of commit order preserving serializable schedules is called **COPSR**.

$$\text{COPSR} \subset \text{OPSR}$$

Comparison of Correctness Criteria

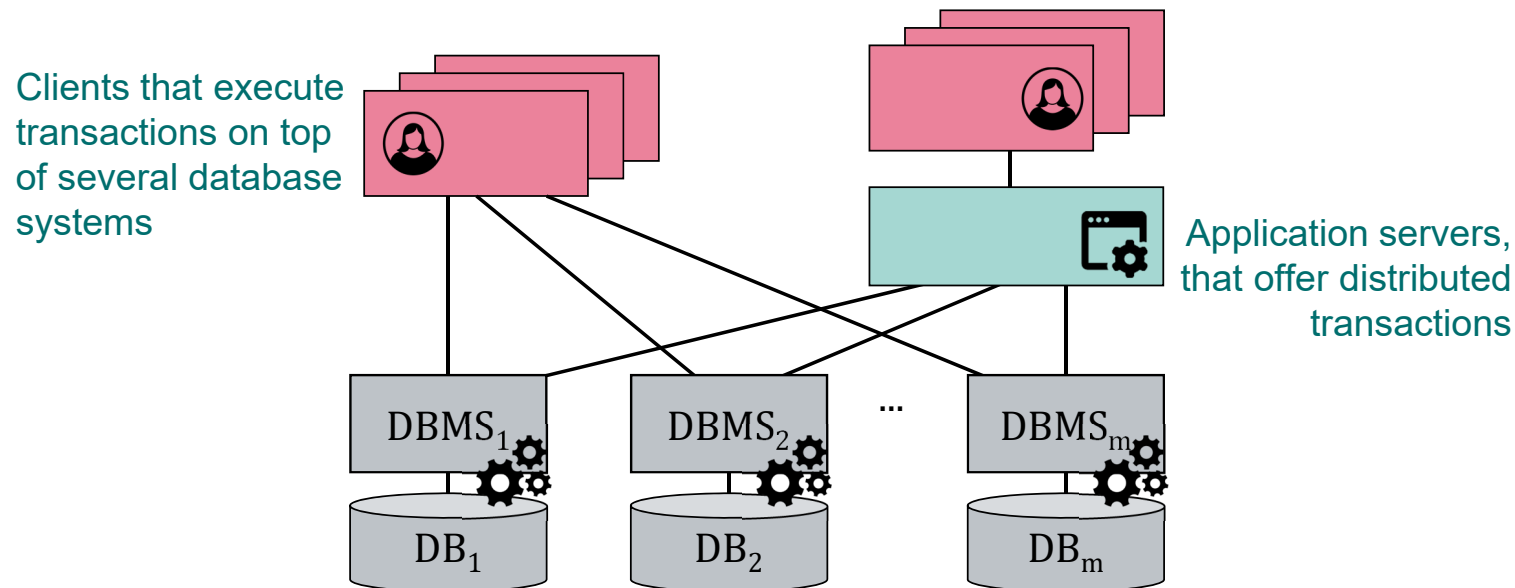


Overview of Chapter 4.2 – Data Consistency

4.1	Self-Organization (P2P)
4.2	Data Consistency (Distributed Transactions)
4.2.1	Foundations of Transaction Management (partly recap from cs246 Database Systems)
4.2.2	Atomicity of Distributed Transactions
4.2.3	Serializability of Distributed Transactions
4.3	Data Replication

Atomicity of Distributed Transactions

- In practice, data are not only stored in a single, centralized database system, but they are rather physically distributed among a set of database systems (also called ‘component databases’)
- However, even in the case single transactions span several systems across the network, **the ACID guarantees of transactions need to be provided globally**



Atomicity in Centralized Databases



Failure Classes

1. Failure in the **application program**
2. Failure in the system software (OS or DBS), leading to a system crash (system failure)
3. Power outage
4. Transient hardware failure
5. Maloperation
6. Disk failure (media failure) → loss of persistently stored data
7. Disaster (fire, earthquake, flood ...)

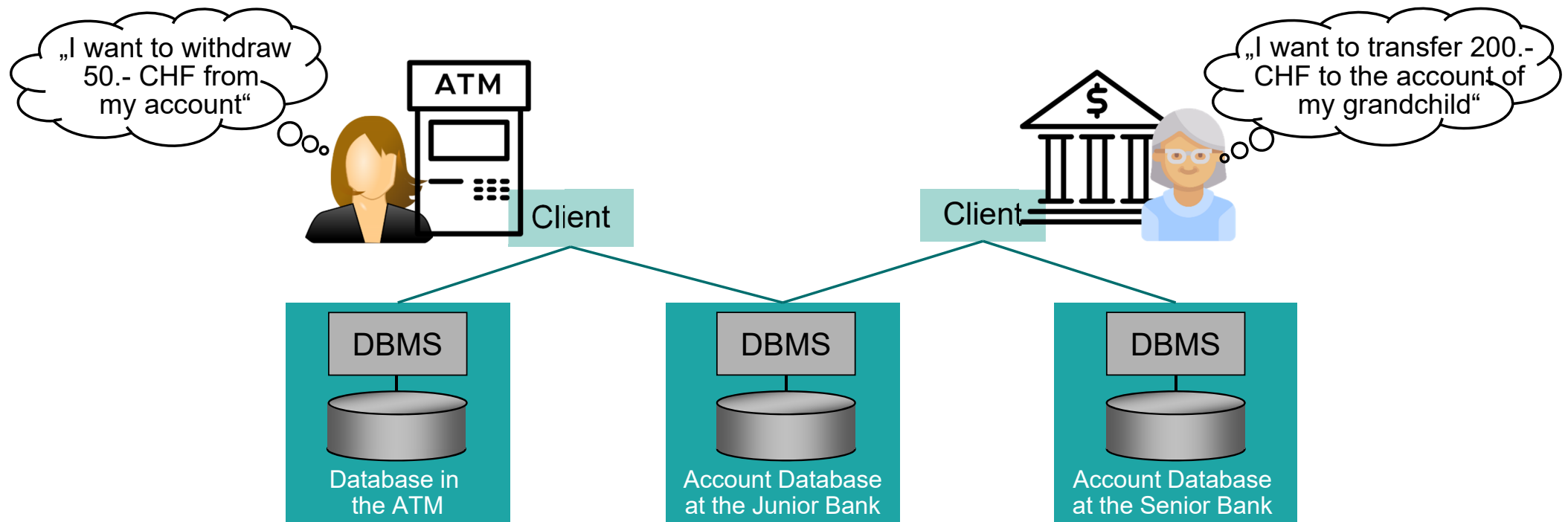
Solution

1. **Abort of a single transaction**
 - Recoverability (RC), Avoiding Cascading Aborts (ACA), Strictness (ST), Rigorousness (RG)
2. – 5. **Crash recovery**
 - consider all active transactions as aborted
 - Redo committed transactions
 - Undo uncommitted transactions
6. – 7. **Disaster recovery** (backup)

Distributed Transactions: An Example

Distributed Databases

- Data are managed in different database systems – transparent to the users. Access to data is location transparent as well.



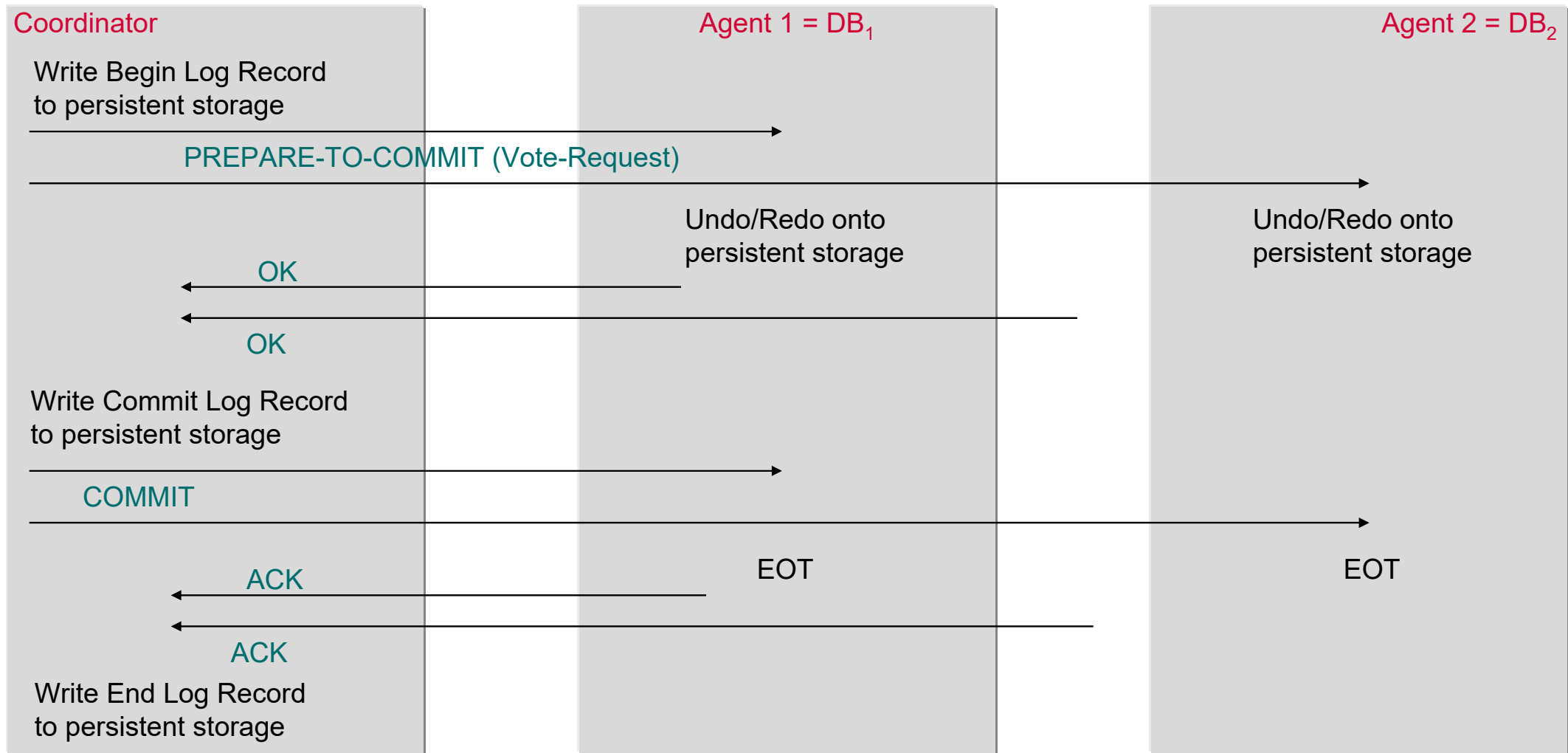
Atomicity of Distributed Transactions

- Usually, the commit in a component database cannot be enforced – since component databases are autonomous (there might be, for instance, violations of integrity constraints in one component database, but also deadlocks, local system crashes, etc.)
- Most importantly, a situation has to be prevented in which one component system writes a commit log for a transaction (this is equivalent to actually performing the commit) while another component database decides to roll back its part of a distributed transaction. **Although atomicity in each subsystem would be provided, this will not be the case for the atomicity of the global transaction!** In addition, different kinds of communication failures (lost messages, duplicates, etc.) have to be taken into account.
- The solution to all this is a dedicated protocol for atomicity in distributed transactions: the **Two Phase Commit Protocol (2PC)**.

Two Phase Commit Protocol (2PC)

- **Basics of 2PC:** One of the systems involved is declared being **coordinator** for the processing of a particular distributed transaction (this is not a static role and can vary from one transaction to the other). For example, the system which has initiated a global transaction can be chosen as coordinator. All other systems are called **agents**. The protocol is defined as follows:
- **1st Phase:** The coordinator contacts all agents and asks them whether they can commit their local sub-transactions. Moreover, it asks them to be arranged for either a subsequent global Commit or a global Rollback ("**Prepare**" message). In the positive case (agent could commit, i.e., there is no local violation of some integrity constraint), all agents are in the state "**prepared**".
- **2nd Phase:** Once having collected the results of the poll among their agents, the coordinator makes a global commit decision and informs all agents about this decision. **Only in case all agents have responded positively, the coordinator is able to come to a global commit decision.** Otherwise, the global decision is "rollback".

Messages and States in 2PC



2PC: Properties ...

- Between answering to the coordinator and the receipt of the final global commit decision, an agent is not allowed to unilaterally decide on its local sub-transaction in question.
- In particular, the agent has to keep all locks for this sub-transaction since the coordinator might come back with a global rollback decision.
 - The agent will be **blocked**

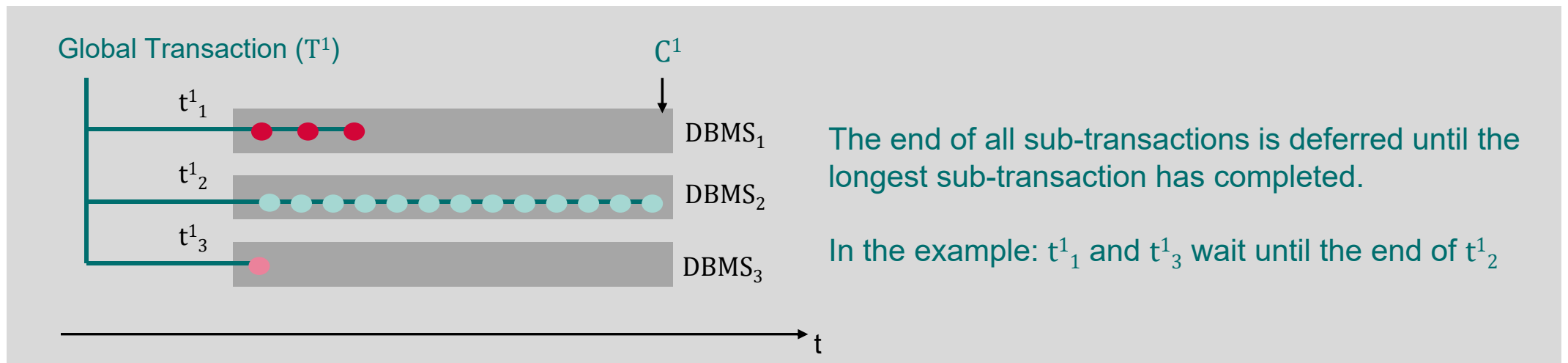
... 2PC: Properties ...

- What happens in case of node failures:

Node which failed	Reaction of Coordinator	Reaction of Agent
Coordinator in phase 1 (before writing commit log record)	After restart: repeat sending of “prepare” message	• If not yet prepared: wait or unilateral abort • If already prepared: wait (blocked), or ask other agents
Coordinator in phase 2 (before writing end log record)	After restart: repeat sending of either “commit” or “abort” message	• Before EOT: wait (blocked), or ask other agents • After EOT: repeat ACK when receiving again commit or abort
Agent in phase 1 (before writing prepared log record)	Repeat sending of “prepare” message	After restart: unilateral abort
Agent in phase 2 (before writing commit log record)	When “ACK” is missing: repeat sending “commit” or “abort”, respectively	After restart: wait for notification (commit or abort) of coordinator, or ask coordinator about decision

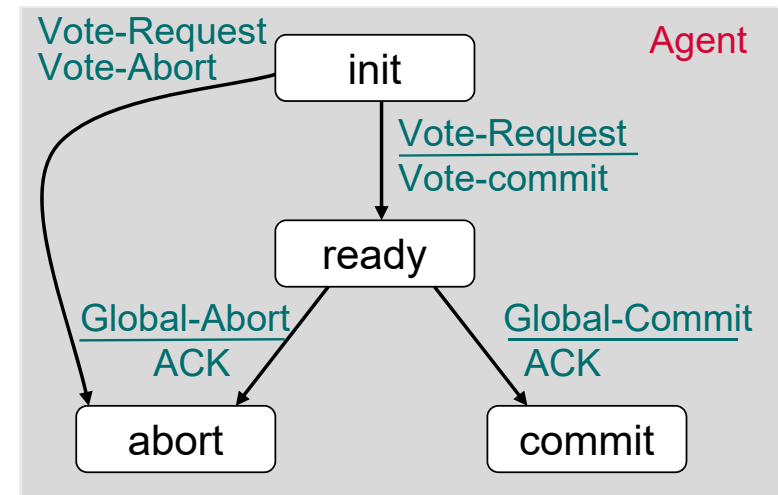
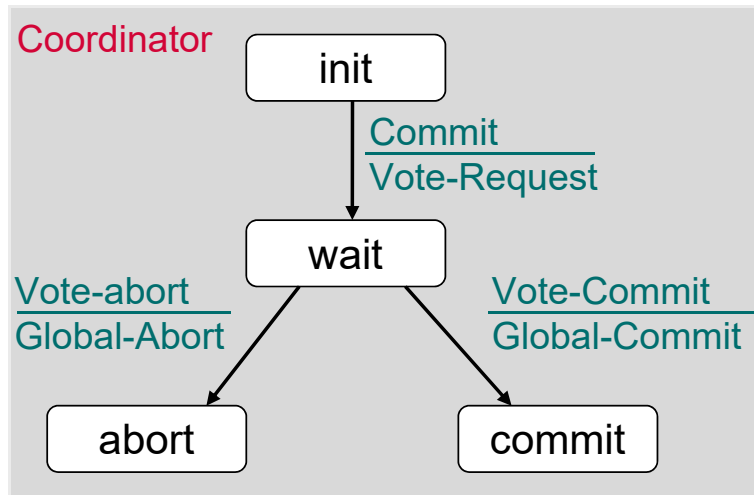
... 2PC: Properties

- Since the coordinator might crash as well or since messages can be lost, the agent can quickly come to a blocking situation which has immediate consequences on the performance of a local system.
- For this reason, 2PC is also called a [blocking protocol](#).



2PC: State Diagram

- In 2PC, blocking is possible since there is a state from which abort as well as commit can be directly reached as successor state (depending on the decision of the coordinator).

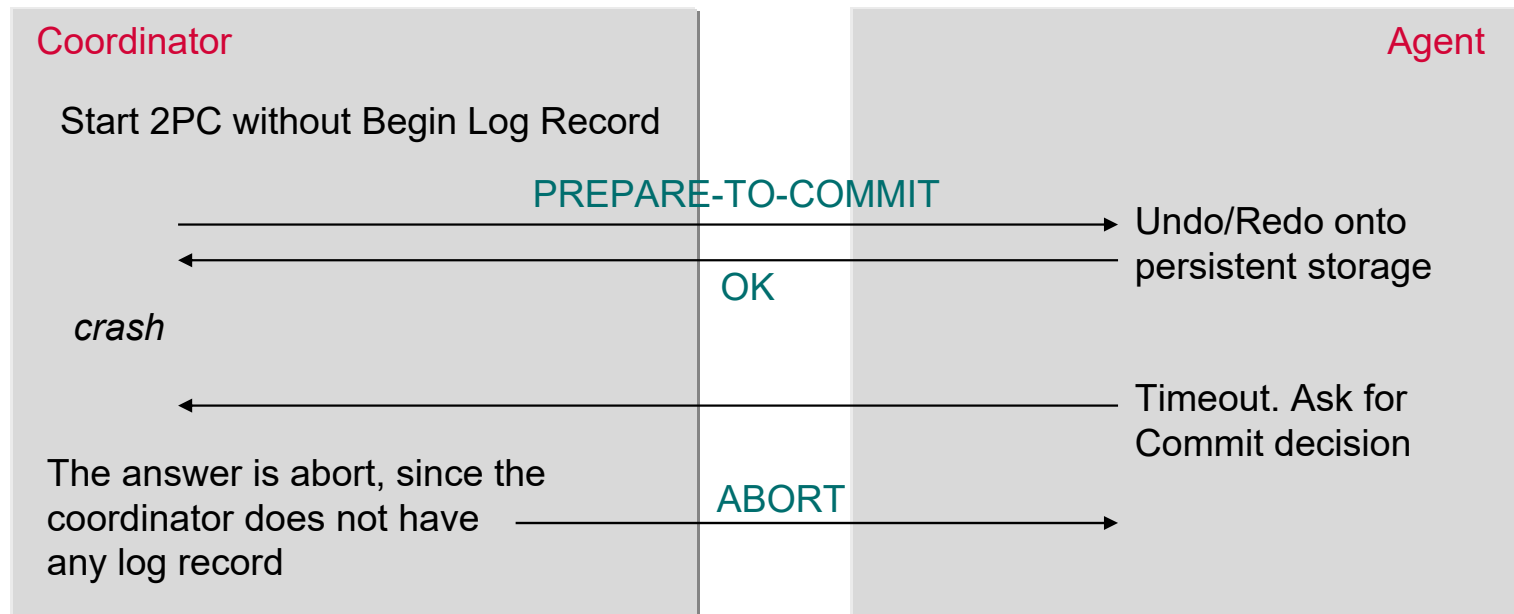


2PC Variants: Presumed Abort ...

- There are very many optimized variants of 2PC which aim at either reducing the messages to be sent or the Log I/Os that need to be done.
- **Presumed Abort 2PC**: In case an agent asks the coordinator for the state of a global transaction, this is mostly done during recovery. In the presumed abort 2PC variant, the coordinator then assumes that the global transaction has failed (unless it has a commit record which states the opposite).
If an agent aborts in the first phase, it writes a local log record accordingly, but it does not notify the coordinator (which assumes a local abort anyway if it is not informed otherwise).

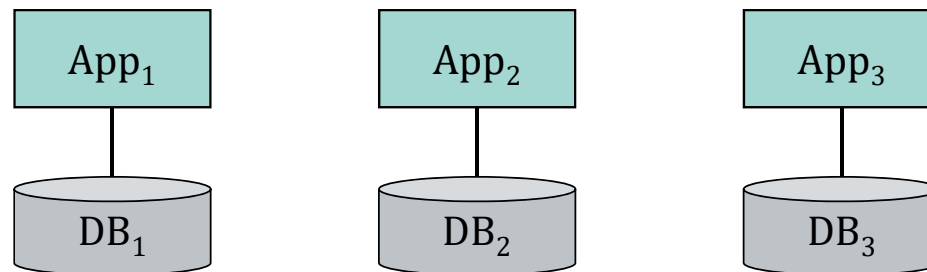
... 2PC Variants: Presumed Abort

- Presumed Abort 2PC (continued)



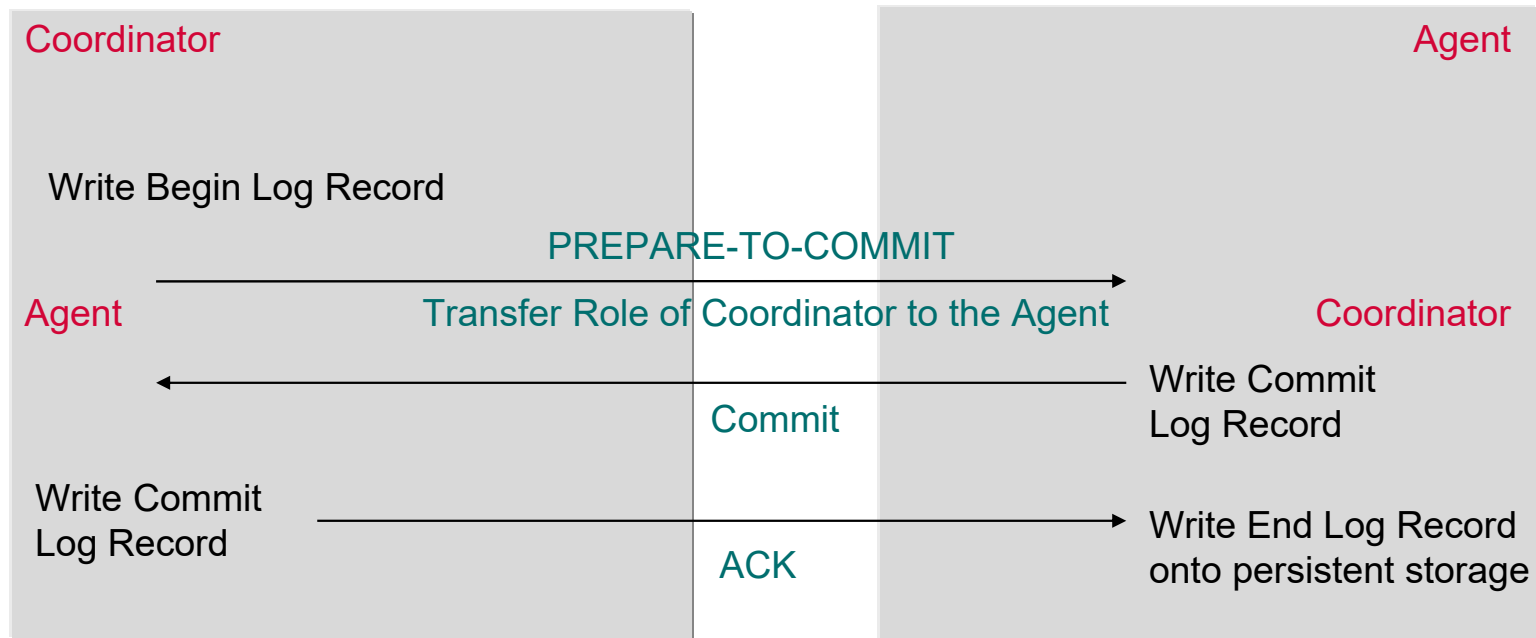
2PC Variants: Transfer of Coordination ...

- **Transfer of Coordination 2PC:** When having one coordinator and one agent, the atomic commitment can be achieved by three interactions (the fourth one is superfluous). Together with the prepare message, the coordinator forwards the coordinator role to the agents (i.e., the roles of both change afterwards). The former agent (now coordinator) goes to prepared state in case of success, locally commits its changes and sends a commit message to the former coordinator. The latter locally commits as well and sends a confirmation to the (new) coordinator.



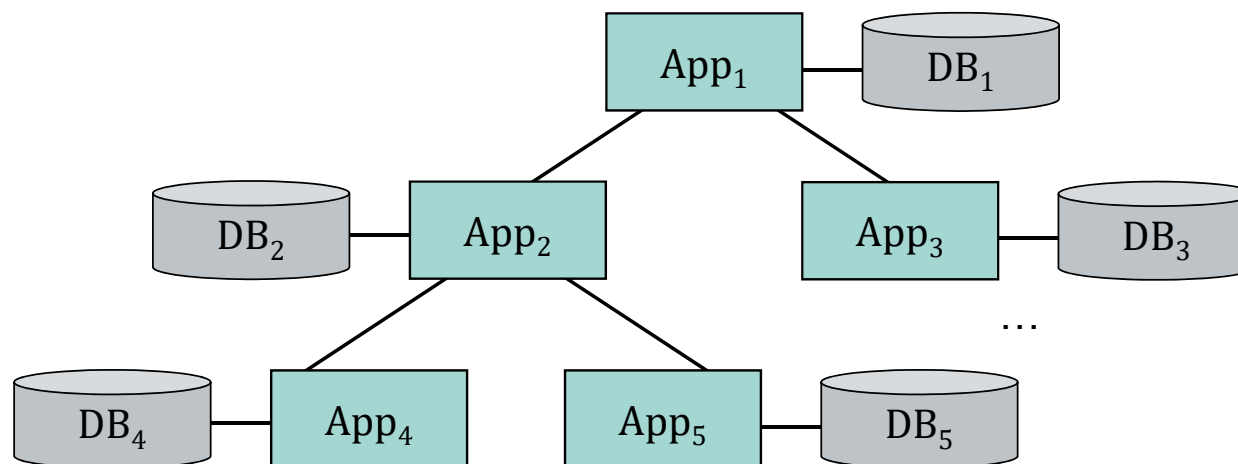
... 2PC Variants: Transfer of Coordination

- Transfer of Coordination 2PC (continued)



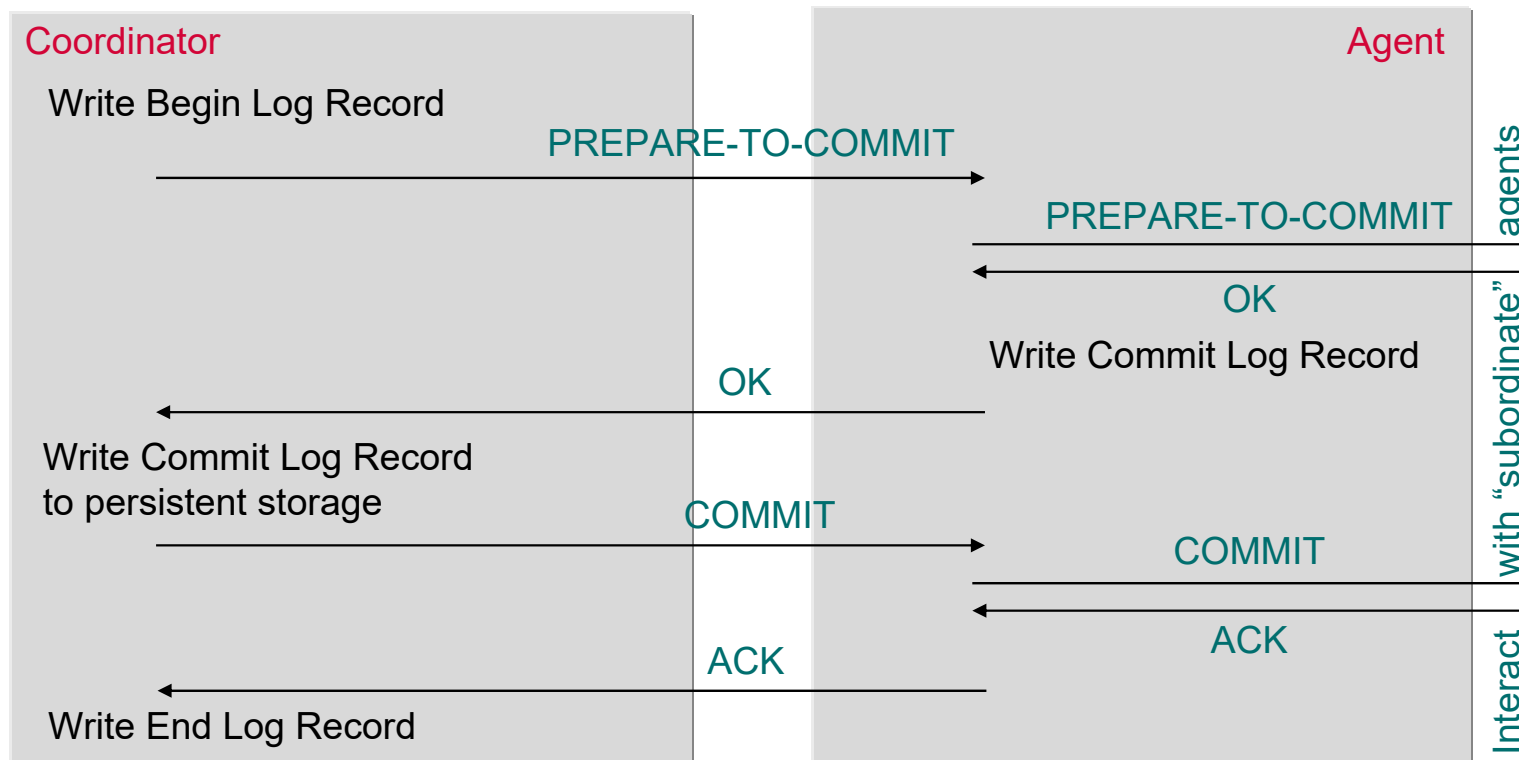
2PC Variants: Tree 2PC ...

- **Tree 2PC** (or **Nested 2PC**): In case the invocation structure is a tree, this structure can also be exploited for the 2PC messages. Usually, the coordinator is the root of the tree, the agents are the inner nodes, and the leaves and the edges correspond to the actual invocations. Then, the coordinator only communicates with its direct successors in the tree and the inner nodes take over the role of “local” coordinators (i.e., they reply to their coordinator only if they and their subordinates are prepared).



... 2PC Variants: Tree 2PC

- Transfer of Coordination 2PC (continued)



3 Phase Commit ...

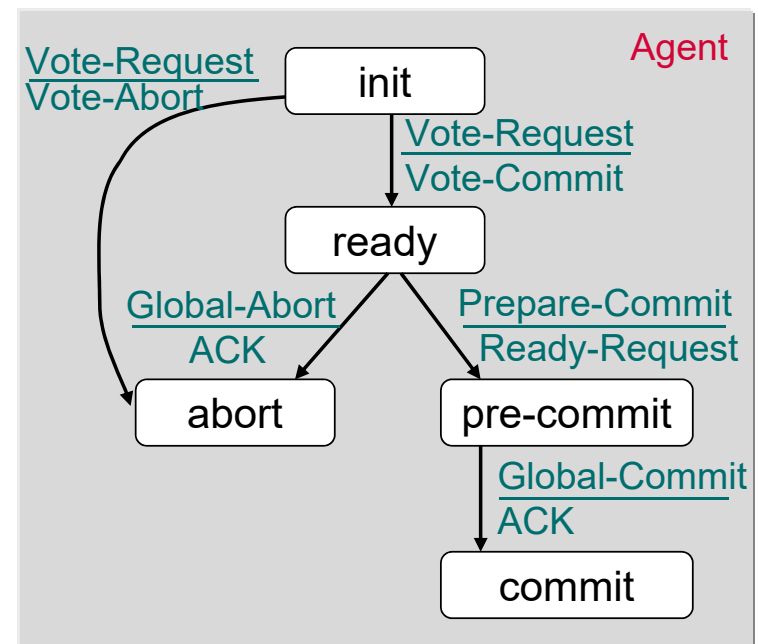
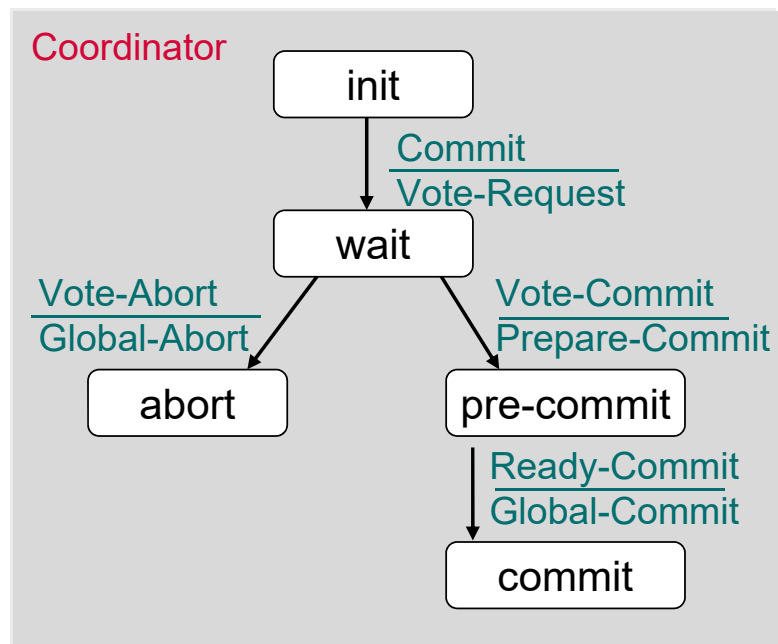
- The goal of the **Three Phase Commit-Protocol (3PC)** is to narrow down as much as possible the phase in which a blocking situation for one (or several) of the agents might occur, e.g., when the coordinator crashes before distributing the commit decision. In this case, in 2PC all agents would have to wait until the coordinator finally recovers.
- In 3PC, there is an additional round of communication between the coordinator and its agents. By this, a new state is generated (“**pre-commit**”) which does only allow to proceed to commit, but not to abort.

... 3 Phase Commit ...

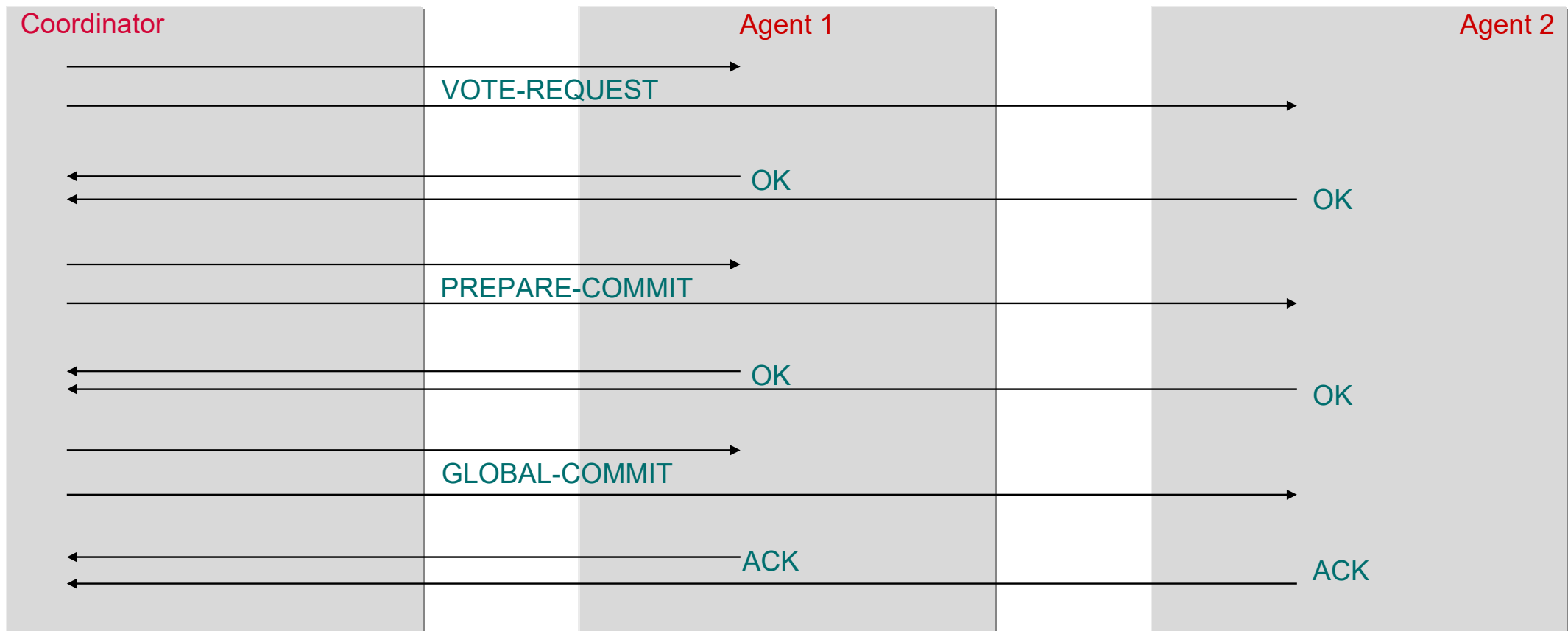
- In detail, 3PC is characterized as:
 - No state which may directly lead to commit or abort (may leave agents in an undecided state, as they do not know whether and how others have already proceeded)
 - No state in which it is not possible to make a final decision and from which a transition to commit can be made

... 3 Phase Commit ...

3PC: state diagrams



... 3 Phase Commit ...



... 3 Phase Commit ...

- Coordinator
 - **1st phase**: If a failure occurs before the start of the commit processing, the coordinator **aborts** the transaction. Otherwise, the coordinator sends a vote-request message to all agents; afterwards, it moves to state **wait**.
 - **2nd phase**: In case of a failure, timeout, or a negative reply from any agent, the coordinator aborts the transaction and sends an abort message to all agents. Otherwise (all agents positively reply before a timeout), it sends **prepare-commit messages** to all agents and moves to state **pre-commit**.
 - **3rd phase**: If all agents acknowledge the prepare-commit message, the coordinator sends a global-commit to all agents. However if the coordinator times out in state **pre-commit** while waiting for an acknowledgement from an agent, it assumes that this agent has crashed – but the agent has before confirmed the vote request. **Therefore, the coordinator can proceed with sending a commit to the remaining operational agents – and rely on a recovery protocol for the crashed agent.**

... 3 Phase Commit

- Agents
 - 1st phase: like phase 1 in the 2PC protocol.
 - 2nd phase: if an agents receives an abort message from the coordinator in state **ready**, it aborts.
If the agent receives a prepare-commit message, it replies with an acknowledgement and awaits a final commit.
 - 3rd phase: the only message that can be received from the coordinator if the agent is in state **pre-commit** is global-commit.

2PC vs. 3PC ...

- In 3PC, an agent can be blocked in state **ready** or in **pre-commit** because of a crash of the coordinator, but ...
- ... in this case, an agent A can contact other agents in order to find out the current status of the overall global transaction.
 - If any of these agents is in state **commit** or **abort**, then A can take this state as well (this is the same for 2PC and 3PC).
 - If all (or the majority of) agents are in state **pre-commit**, then they can safely commit, even in case the coordinator does not recover.
 - If any other agent is in state **init**, then the transaction can be safely aborted (this means that none of the agents can be in **pre-commit**). Again, this holds for both 2PC and 3PC.
 - If all agents (or at least the majority of agents) are in state **ready**, then they can decide to abort the global transaction. This guarantees correctness, even if in addition to the coordinator also other agents in state **pre-commit** have crashed.

... 2PC vs. 3PC

- This case in which the coordinator and at least one of the agents has crashed forms the major difference between 2PC and 3PC.
 - In 2PC, the crashed agent could be in state commit so that the others are not able to take any decision and will be blocked until the coordinator returns.

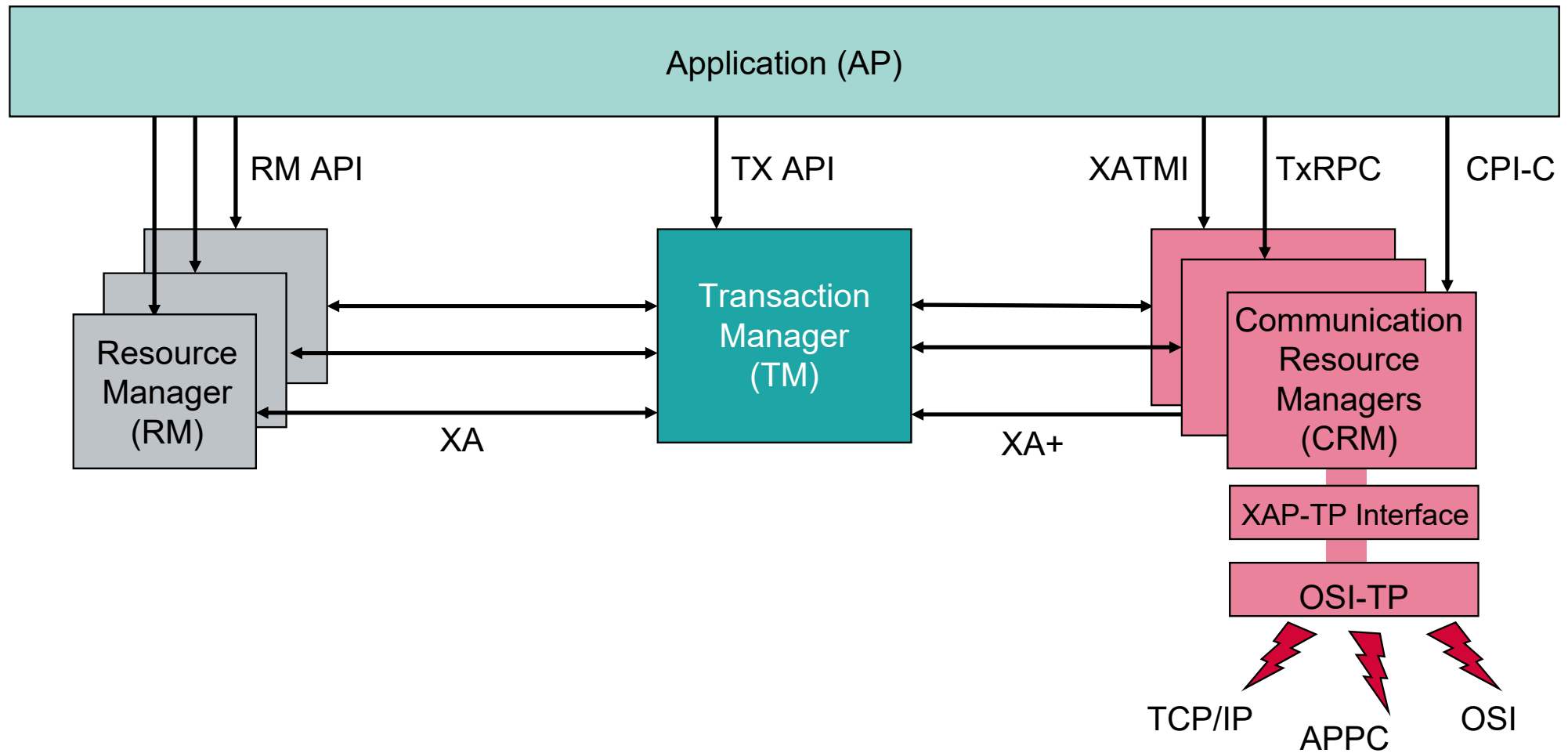
2PC: Implementation

- The 2PC protocol is standardized ([ISO/OSI](#), [X/Open](#)).
- 2PC are supported by more or less all commercial DBMS which can act as both coordinator and agent.
- In addition, the coordinator role of an X/Open 2PC distributed transaction can be taken over by various middleware services.

Distributed Transactions: X/Open DTP

- The X/Open DTP (Distributed Transaction Processing) standardizes APIs for the development of distributed transactions and is based on the two phase commit protocol
 - Often also called **XA protocol**
 - X/Open DTP distinguishes several components
 - RM: Resource Manager, e.g., a DBMS
 - TM: Transaction Manager
 - CRM: Communications Resource Manager
- The standard uses different interfaces
- For the interaction between the application program and the different X/Open DTP components
 - For the interaction among the X/Open DTP components

X/Open DTP: Overall Architecture



X/Open DTP: APIs ...

Interfaces to the application

- RM API = Resource Manager Application Program Interface
 - Interface between application program and resource manager (= database)
 - e.g., ESQL, CLI (JDBC is, for example, such a CLI)
- TX API = API between application program and Transaction Manager (TM)
 - Specification of transaction boundaries: **BeginTX**, **CommitTX**, **AbortTX**
- Transactional communication to invoke application services in a transaction context. For this, X/Open DTP makes use of some existing APIs
 - TxRPC (Transactional Remote Procedure Call) – based on RPC
 - XATMI: XA Transaction Monitor Interface
 - CPI-C (Common Programming Interface for Communications). This is an API that supports peer-to-peer-style of communication between application programs

... X/Open DTP: APIs

- XA API: interface between RM and TM
 - Supports the actual 2PC processing (**PrepareCommit**, **Commit**, **Rollback**)
- XA⁺ API: extension of XA that allows a CRM to register a further (subordinate) node with the TM that participates to the distributed transaction
- OSI-TP: for the communication between heterogeneous transaction managers

Transactional Client/Server Communication

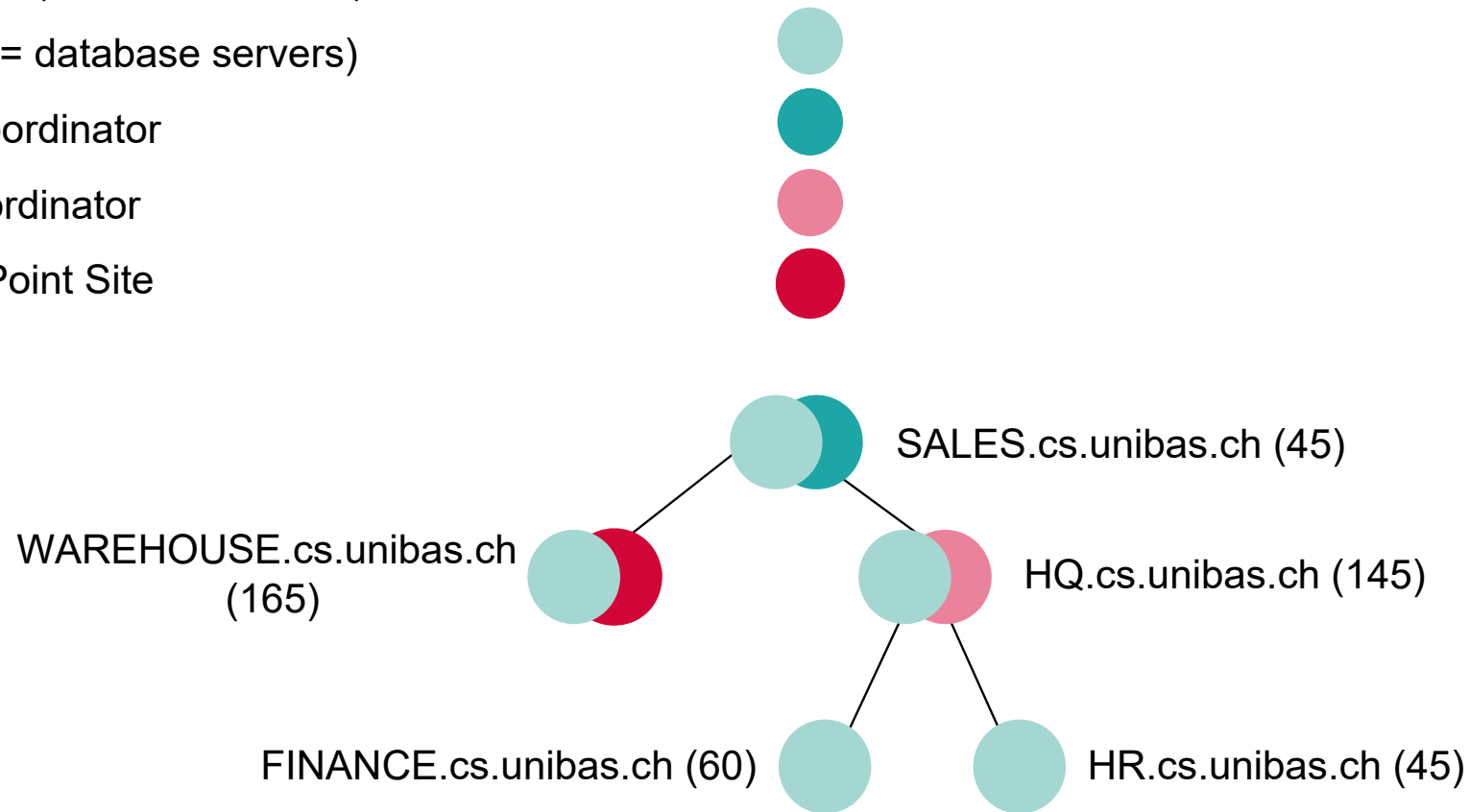
- TxRPC: Transactional Remote Procedure Call: Conventional RPC, plus a call becomes a (sub-)transaction of the global distributed client transaction
 - An automatically generated transaction ID (TXID) is assigned to each transaction
 - Each call (or each message) contains –in addition to the actual request parameters– the transaction's TXID
 - This TXID allows to ...
 - Coordinate all operations of a distributed transaction, i.e., to derive a consensus on a joint commit or rollback (2PC protocol), since they can immediately be identified as belonging to the same transaction
 - Guarantees that messages are received exactly once or that services are invoked exactly once (“exactly-once” semantics)

Example: 2PC in Oracle Databases

- In distributed (homogeneous Oracle) database environments, Oracle automatically applies a **tree 2PC protocol** (i.e., a modified version of 2PC)
- **Global coordinator**: database server that initiates a distributed transaction (first call via DB link)
- **Local coordinator**: database server ($\neq$ global coordinator), that again references data in a remote database server (nested 2PC)
- **Commit Point Strength**
 - Each node (= database server) is assigned, during configuration, a weight in the interval $[0 \dots 255]$ – the so-called “**Commit Point Strength**” (this is done by the database admin at the time, the database is set up)
- **Commit Point Site**
 - Node with the highest weight
 - Does *not* need to enter the prepared state (why is that an issue at all?)

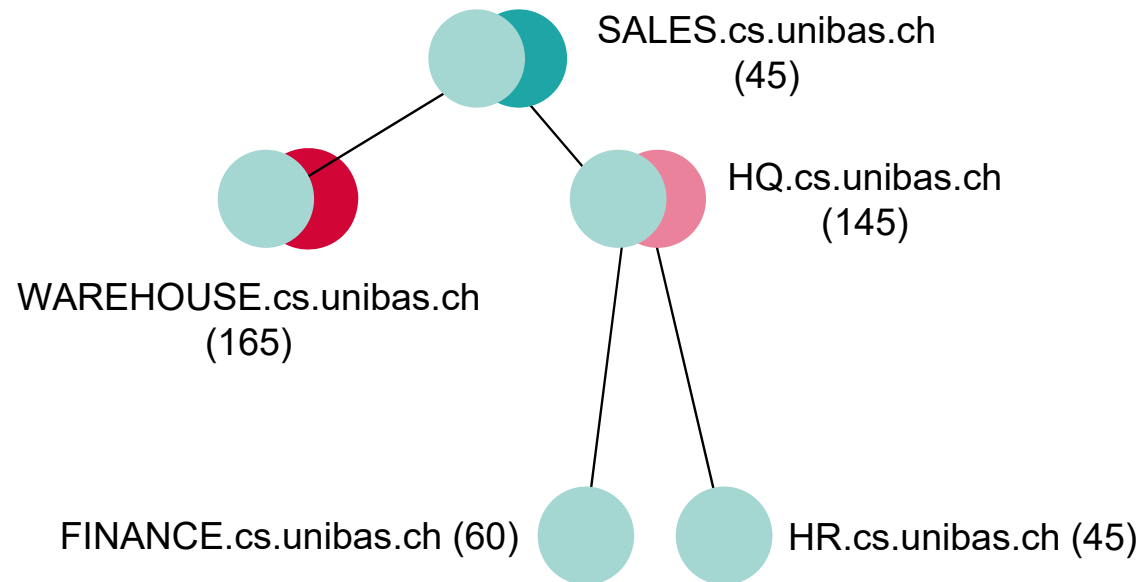
Oracle 2PC – Sample Execution ...

- Configuration (“transaction tree”)
 - n nodes (= database servers)
 - Global coordinator
 - Local coordinator
 - Commit Point Site



... Oracle 2PC - Sample Execution ...

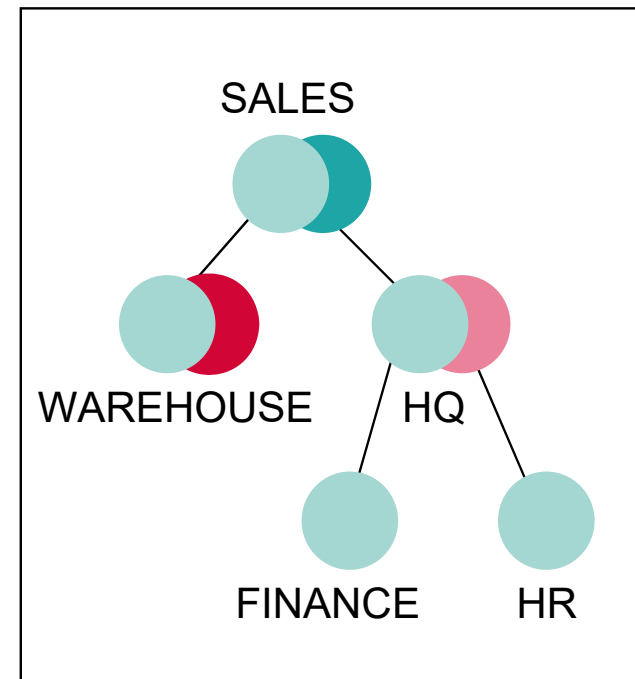
```
BEGIN
  INSERT INTO orders o VALUES ...;
  EXECUTE admin.man@HQ.cs.unibas.ch;
  UPDATE wh@WAREHOUSE.cs.unibas.ch SET ...;
COMMIT;
```



```
CREATE PACKAGE BODY admin
  AS (
    PROCEDURE man ( ...
    BEGIN
      UPDATE balance@FINANCE...
      SET ...;
      INSERT INTO management
      VALUES ...;
      UPDATE employees@HR...
      SET ...;
      Commit;
    END man;
    ...)
```

... Oracle 2PC - Sample Execution

- Global coordinator sends **prepare** message to (n-1) nodes (without Commit Point Site)
 - Here: HQ & SALES
- Local coordinator (HQ) sends **prepare** to FINANCE & HR
- On global commit decision, a **commit** is issued at the Commit Point Site (WAREHOUSE)
- After commit in WAREHOUSE, the **commit** of the overall distributed transaction is initiated, otherwise its **rollback**



Oracle 2PC – Summary

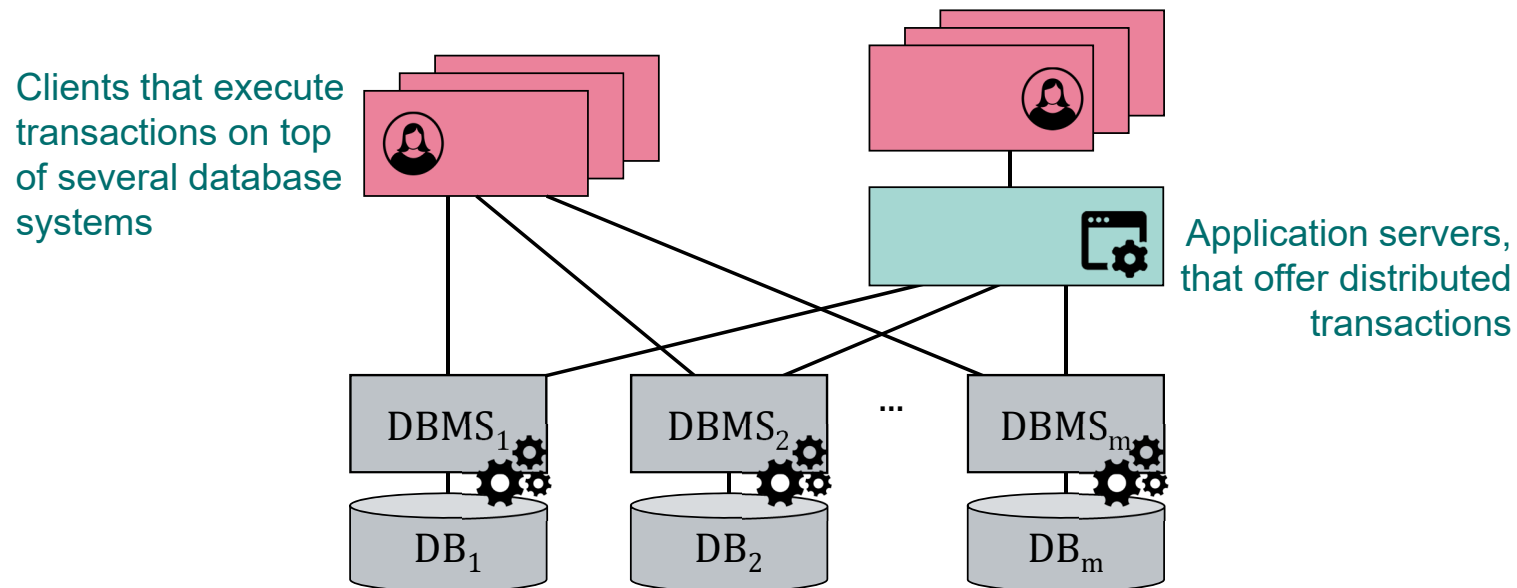
- Transparent application of 2PC in distributed databases (even following a so-called “TP-Lite approach”, i.e., having all application logic residing inside the database)
- Distributed transactions are even supported in heterogeneous environments
 - DB-Link may also be established to non-Oracle database servers
 - Prerequisite: transparent transformation between different SQL dialects by means of specialized “agents”. These have to be provided for each non-Oracle system that needs to be included
 - Support in distributed transactions depends on the properties of the ‘external’ system
 - Prerequisite for 2PC: implementation of X/A Interface
 - Otherwise: only participation as Commit Point Site or only as read-only-site, respectively

Overview of Chapter 4.2 – Data Consistency

4.1	Self-Organization (P2P)
4.2	Data Consistency (Distributed Transactions)
4.2.1	Foundations of Transaction Management (partly recap from cs246 Database Systems)
4.2.2	Atomicity of Distributed Transactions
4.2.3	Serializability of Distributed Transactions
4.3	Data Replication

Serializability of Distributed Transactions

- Global transactions, i.e., transactions that span several database systems, also need to be shielded from each other –at global level– in the same way different transactions inside one DBMS are serialized (= isolation property).



Multi Database Systems (MDBMS)

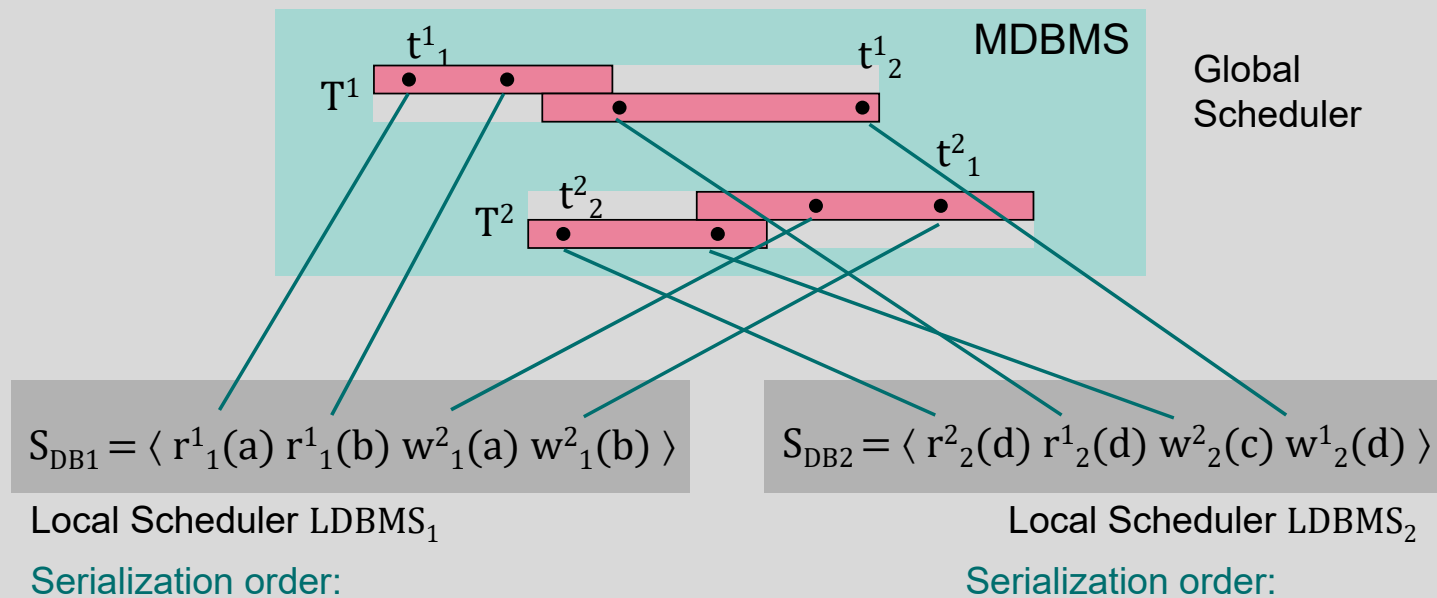
- Ideally, a distributed database system consists of a set of m cooperating and homogeneous database systems. All transactions are global, i.e., are sent to the schedulers of the local component database systems via a global scheduler.
- These distributed databases are also called **Multi Database Systems (MDBMS)**.
- **Global Transactions T^i** (which are initiated by clients or application programs) consist of a set of **sub-transactions t_k^i** , one per component DBMS:
 - $T^i = \langle t_1^i, t_2^i, \dots, t_m^i \rangle$ This means that global transactions are split into smaller parts (projected onto the individual underlying databases).
 - Each sub-transaction is –fully in line with the basic transaction model – a sequence of database actions.
 - We assume that there is a **partial** order between the sub-transactions of a transaction. This means that certain sub-transactions can be executed in parallel.

Example: MDBMS Transactions

Example:

$T^1 = \langle t^1_1 t^1_2 \rangle$ with $t^1_1 = \langle r^1_1(a) r^1_1(b) \rangle$ $t^1_2 = \langle r^1_2(d) w^1_2(d) \rangle$

$T^2 = \langle t^2_1 t^2_2 \rangle$ with $t^2_1 = \langle w^2_1(a) w^2_1(b) \rangle$ $t^2_2 = \langle r^2_2(d) w^2_2(c) \rangle$





Recap: Two-Phase Locking (2PL)

- Main idea:
 - Before an object is accessed (either in read or write mode), a **lock** has to be acquired. These locks have to make sure that no two conflicting actions can be executed on the same object.
 - These locks have to be managed (acquired, checked, released) by the system in a way that is **fully transparent to the application and/or user**.

- Example

The transaction $T^1 = \langle a^1_1 a^1_2 \dots a^1_k \rangle$

will be automatically transformed into:

$T^{1*} = \langle L(a^1_1) a^1_1 L(a^1_2) a^1_2 \dots L(a^1_k) a^1_k U(a^1_1) U(a^1_2) \dots U(a^1_k) \rangle$

where $L(a^i_k)$ acquires a lock on the object accessed by action a^i_k („lock“)
 $U(a^i_k)$ releases the lock held for a^i_k („unlock“)



Recap: Lock Modes and Compatibility

- The objective of the 2PL protocol is to **avoid conflicting actions on the same object**, but at the same time **not to constrain actions that are not in conflict**. Hence, two parallel transactions might read the same object, but only one is allowed to write (and then, no additional read is allowed).
- For this, two different lock modes are needed:
 - **S lock (= Shared Lock)** for read access $r(x)$
 - **X lock (= Exclusive Lock)** for write access $w(x)$
- Locks are a kind of semaphores, implemented by the DBS. The lock mode will be chosen automatically by the DBS. For the two modes, the following lock compatibility matrix can be defined:

		Lock acquired	
		Shared	eXclusive
Lock held	Shared	+	—
	eXclusive	—	—

- + **acquired lock can be granted** (execution order of two readers can be changed; no conflict in case of r/r)
- **acquired lock CANNOT be granted** (because of w/r , r/w , or w/w -conflict). The requesting transaction has to wait).

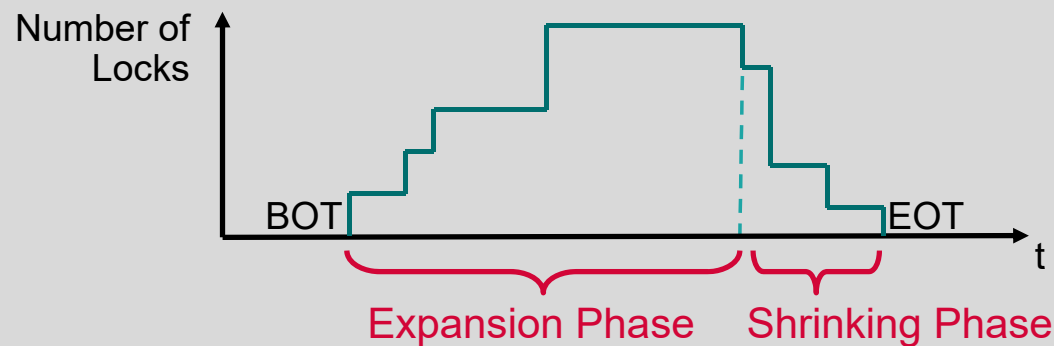


Recap: 2PL Protocol

A scheduler follows the **2PL protocol** if it:

- (1) acquires a (compatible) lock $L(a_m^i)$ before each action a_m^i
(L = shared when a_m^i is a read, otherwise: L = exclusive)
- (2) holds the lock $L(a_m^i)$ at least until action a_m^i has been executed
- (3) after a lock is released, no other lock of the same transaction can be acquired.

- A **2PL scheduler only produces serializable (CPSR) schedules**; actually, the resulting schedules of 2PL are even **order-preserving serializable (OPSR)**.



Recap: 2PL Protocol – Remarks



- Rule (1): enforces that conflicting actions are in the order of the corresponding lock requests. If a lock request for action a_m^i cannot be granted, then transaction T^i has to wait (and of course all subsequent actions of T^i that are ordered after a_m^i).
- Rule (2): enforces that actions are actually executed in the order determined by the scheduler.
- Rule (3): lock request and the release of locks is separated into two phases:
in the **expansion phase**, a transaction acquires locks. With the first lock that is released, a transition to **the shrinking phase** is done; from this time onwards, no additional lock can be requested/acquired.



Recap: Strict Two-Phase Locking (S2PL) Protocol

- The 2PL protocol allows locks to be released „early“, i.e., before the commit of the corresponding transaction. However, as an on-line scheduler can only decide on the basis of the prefix of a schedule, the early release of locks can be risky.
- Remedy: all locks have to be kept until commit time.

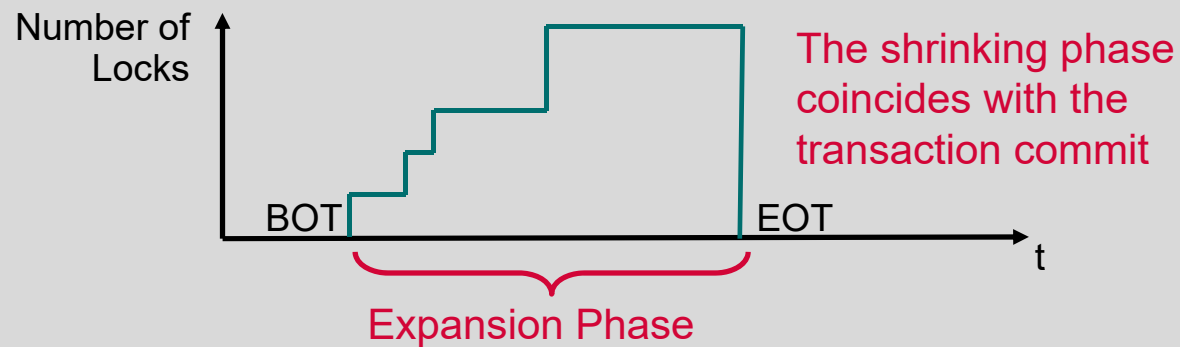
A scheduler follows the strict two-phase locking (S2PL) protocol if it:

- (1) acquires a lock $L(a_m^i)$ before each action a_m^i (S for reads, X for writes)
- (2) all locks $L(a_m^i)$ are held until the end of its transaction (i.e., they will all be jointly released at commit time)

Recap: S2PL



- Apparently, S2PL is stronger than 2PL. Hence, each S2PL protocol also guarantees serializable executions. In addition, each schedule produced by an S2PL scheduler is even **commit-order preserving serializable (COPSR)**.



Deadlock Avoidance – Transaction Priorities

- Lock-based protocols might lead to deadlock situations (cyclic wait dependencies) when a transaction T^k does not get a lock because it is already held by another transaction T^i .
- Each transaction T^i is associated with a **priority** $P(T^i)$.
 - Depending on the priority of the transaction requesting a lock and of the transaction holding a lock, the requesting transaction either waits or gets aborted.
 - Priorities can be assigned using timestamps: each transaction receives a unique timestamp $ts(T^i)$, usually the BOT time. Then, priorities can be defined as follows:

$$P(T^i) = \frac{1}{ts(T^i)}$$

- Hence, old transactions have a higher priority as younger transactions.

WAIT-DIE and WOUND-WAIT Strategies

- Based on transaction priorities (timestamps), two strategies for avoiding deadlocks can be defined (“wild west” methods).
- If the lock request of a transaction T^k is not granted because an incompatible lock is held by another transaction T^i , then depending on the priorities the following strategies can be identified:



WAIT-DIE:

- If $P(T^k) > P(T^i)$: T^k waits
- Otherwise ($P(T^k) < P(T^i)$): T^k dies (is aborted)
- Hence, T^k waits for younger transactions.

WOUND-WAIT:

- If $P(T^k) > P(T^i)$: T^i is “wounded”, i.e., has to be aborted
- Otherwise ($P(T^k) < P(T^i)$): T^k waits
- Hence, T^k waits for older transactions and “shoots” younger transactions.

WAIT-DIE and WOUND-WAIT: Properties

- Both strategies are unnecessarily restrictive since they assume a deadlock with each lock request that cannot be granted (which however is not always the case). Hence, always one of the involved transactions is aborted.
- In none of the two strategies the older of the conflicting transactions is aborted.
- **WAIT-DIE:**
 - The older a transaction, the more likely it has to wait.
 - Infinite waits can be prevented when a lock that is released is always granted to the waiting transaction with the highest priority.
- **WOUND-WAIT:**
 - The abort of younger transaction can lead to starvation: a transaction is aborted, gets newly started, be aborted again due to the same lock conflicts, and so on. This is also called a “livelock”.
 - Starvation can be avoided when an aborted transaction keeps its original timestamp after restart.

Ordered Shared Locks (OSL)

- 2PL (and its various variants) guarantee CPSR, but might discard schedules that are actually correct.
- The reason for this is that conflicting actions have so far been strictly separated and thus ordered by exclusive locks.
 - **S lock**: no constraint / restriction on the order of actions (there is no conflict between two readers)
 - **X lock**: if T^k holds an X lock on a data object a , another transaction T^i must wait until T^k releases the X lock. Hence, conflicting actions are executed serially (only one transaction has an X lock at a time)

Example: $S_2 = \langle w^1(x) \ r^2(x) \ r^3(y) \ C^3 \ w^1(y) \ C^1 \ r^2(y) \ C^2 \rangle$

S_2 is in CPSR, but cannot be generated by 2PL, since T^1 has to release its X lock on x for T^2 , but it cannot acquire any other lock afterwards, in particular not one for y .

Locks with an Order

- Basic idea of Ordered Shared Locking:

Allow parallelism even in case of conflicts, but respect the order of the locking requests also for conflicts!

- For this, an additional lock mode is needed:
 - **S Shared:** like in (S)2PL – unlimited parallelism
 - **X eXclusive:** like in (S)2PL
 - **OS Ordered Shared:** parallelism is possible, but constrained
- Locks on the same object can be held in parallel, but an order $\rightarrow$ is associated with the lock requests:
 $L^i(x, OS) \rightarrow L^k(x, OS)$ if T^i and T^k concurrently hold locks on x with T^i being first, i.e., $L^i(x, OS) < \dots < L^k(x, OS) < \dots < U^i(x)$ is possible.
- This order also needs to be kept when executing conflicting actions (a^i, a^k) . Hence, **conflict pairs have to be executed in the order given by the lock requests: if $L^i(x, OS) \rightarrow L^k(x, OS)$ then also $a^i(x) < a^k(x)$ has to hold.**

Lock Compatibility Matrix in OSL

- OS locks can be considered as alternative to X locks for write accesses.
- This leads to the following **lock compatibility matrix**:

			Transaction T^k requests lock for		
			Read	Write	
			Shared	Ordered Shared	eXclusive
Transaction T^i holds lock for:	Read	Shared	+	→	—
	Write	Ordered Shared	→	→	—
		eXclusive	—	—	—

+ locks are compatible

— locks are NOT compatible

→ Locks are compatible when order is preserved

Ordered Shared Locking

- The lock compatibility matrix guarantees that each conflict pair is ordered in the lock order. However, it does not (yet) guarantee that the order of all conflict pairs is the same.

Example: $S_3 = \langle w^1(x) r^2(x) w^2(y) r^1(y) C^2 C^1 \rangle$

S_3 is not in CPSR, but would be possible with OS locks
→ this means additional rules will be necessary.

- A transaction T^k depends on T^i (“ T^k has locks on hold”), if
 - T^k holds an OS lock and T^i has acquired before a lock (S or OS) on the same object, but has not yet released this lock, or
 - T^k holds an S lock and T^i has acquired before an OS lock that is not yet released

OSL Rules

OSL rules: A scheduler conforms to the OSL protocol if:

- All transactions respect the two-phase nature of 2PL
- A dependent transaction does not release any locks

- In other words, the important second rule states that if T^k wants to commit, all transactions T^i on which T^k depends must be committed (or aborted) beforehand.

OSL: Possible Protocols ...

- The OSL rules simply require that the two-phase properties are respected, that writers either request an X or an OS lock, and that dependencies are preserved when locks are released
- Still, it is open when OS and when X locks can (have to) be selected
- This leads to a family of in total 8 different protocols, $P_1 - P_8$, of which P_8 is the most common ordered shared locking protocol

... OSL: Possible Protocols

P ₁	r	w
r	S	X
w	X	X

= 2PL

P ₂	r	w
r	S	X
w	OS	X

P ₃	r	w
r	S	OS
w	X	X

P ₄	r	w
r	S	X
w	X	OS

P ₅	r	w
r	S	OS
w	OS	X

P ₆	r	w
r	S	OS
w	X	OS

P ₇	r	w
r	S	X
w	OS	OS

P ₈	r	w
r	S	OS
w	OS	OS

OSL: Examples

Example 1: $S_4 = \langle w^1(x) r^2(x) r^3(y) C^3 w^1(y) C^1 r^2(y) C^2 \rangle$
 when using OSL protocol P_8 , the schedule with lock requests is:

$S_4 = \langle L^1(x, OS) w^1(x) L^2(x, S) r^2(x) L^3(y, S) r^3(y) U^3(y) C^3 L^1(y, OS) w^1(y) U^1(x, y) C^1 L^2(y, S) r^2(y) U^2(x, y) C^2 \rangle$

↑
 T^1 is not dependent

↑
 T^2 is also not dependent

P_8	r	w
r	S	OS
w	OS	OS

Example 2 (also using protocol P_8):

$S_5 = \langle w^1(x) r^2(x) r^3(y) C^3 w^1(y) r^2(y) C^2 C^1 \rangle$

$S_5 = \langle L^1(x, OS) w^1(x) L^2(x, S) r^2(x) L^3(y, S) r^3(y) U^3(y) C^3 L^1(y, OS) w^1(y) L^2(y, S) r^2(y) U^2(x, y) \rangle$

← Impossible! Since T^2 depends on T^1 , T^2 has to wait for the commit of T^1

OSL: Properties

- OSL protocol P_8 that does not use any exclusive locks, additionally also guarantees order preserving serializable schedules (OPSR).
- If, in addition, P_8 is extended by a strict two phase lock request and release procedure, even commit order preserving schedules (COPSR) are guaranteed.
- The strict two-phase P_8 protocol even allows all possible COPSR schedules.

MDBMS: Correctness

- The conclusion from the preceding example is that there must be a guarantee that **all pairs of conflicting operations are globally ordered in the same way**, i.e., that all local serialization orders coincide.
- For this, it is important to know about all local serialization orders or even better to impose them to the underlying DBMSs.
- Hence, there is a need for protocols that support **commit order preserving serializable (COPSR)** schedules.
- **S2PL (strict two-phase locking)** and ordered shared locking are protocols where the commit order and the serialization orders coincide.

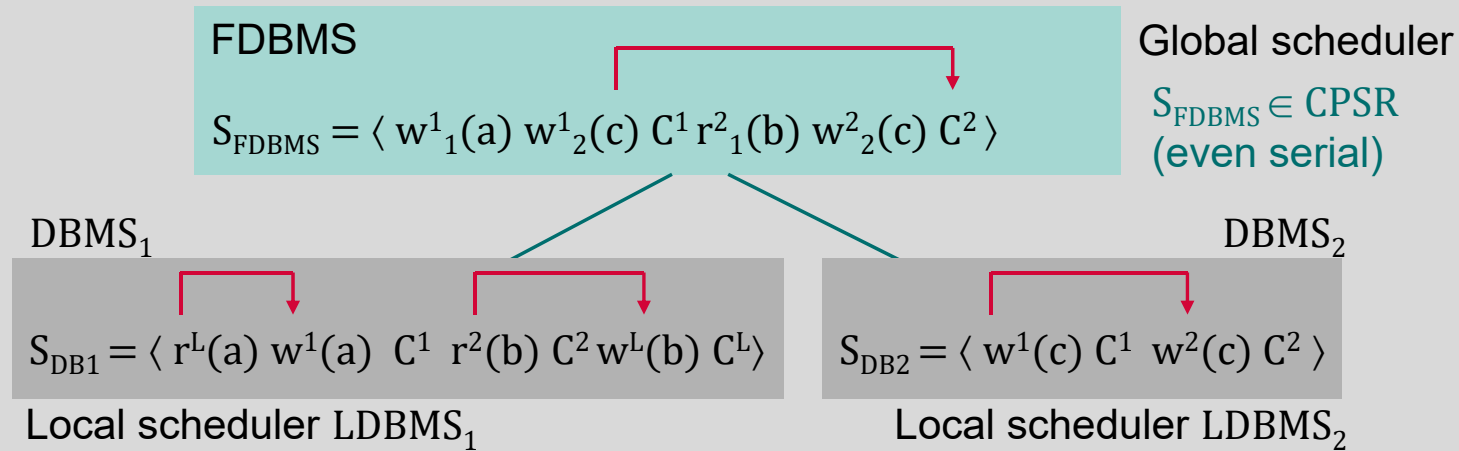
⇒ **Prerequisite for correctness in MDBMS is that each component system applies an S2PL protocol.**

Federated DBMS (FDBMS)

- Federated database systems (FDBMS) have a structure quite similar to that of MDBMSs.
- The most important differences are:
 - In FDBMSs, the component systems are not necessarily homogeneous
 - All FDBMS component systems are autonomous; this means there might be local transactions in addition to the local sub-transactions of a global transactions. These local transactions are not known to the global scheduler.
- This has immediate consequences on correctness since it is no longer globally possible, in each case, whether or not an execution is correct.

Example: FDBMS Transactions

Example: $T^1 = \langle t^1_1 t^1_2 \rangle$ with $t^1_1 = \langle w^1_1(a) \rangle$ $t^1_2 = \langle w^1_2(c) \rangle$
 $T^2 = \langle t^2_1 t^2_2 \rangle$ with $t^2_1 = \langle r^2_1(b) \rangle$ $t^2_2 = \langle w^2_2(c) \rangle$
 $T^L_1 = \langle r^L_1(a) w^L_1(b) \rangle$ local transaction in $DBMS_1$



FDBMS: Correctness

- The MDBMS problems are much more severe in federated database, since it is no longer possible to decide globally, without local knowledge, about correct executions.
- However, it finally turns out that the solution to the problem is still the same: by having a chance to enforce a certain serialization order from the outside by using a COPSR protocol, also the effects of purely local transactions can be controlled.

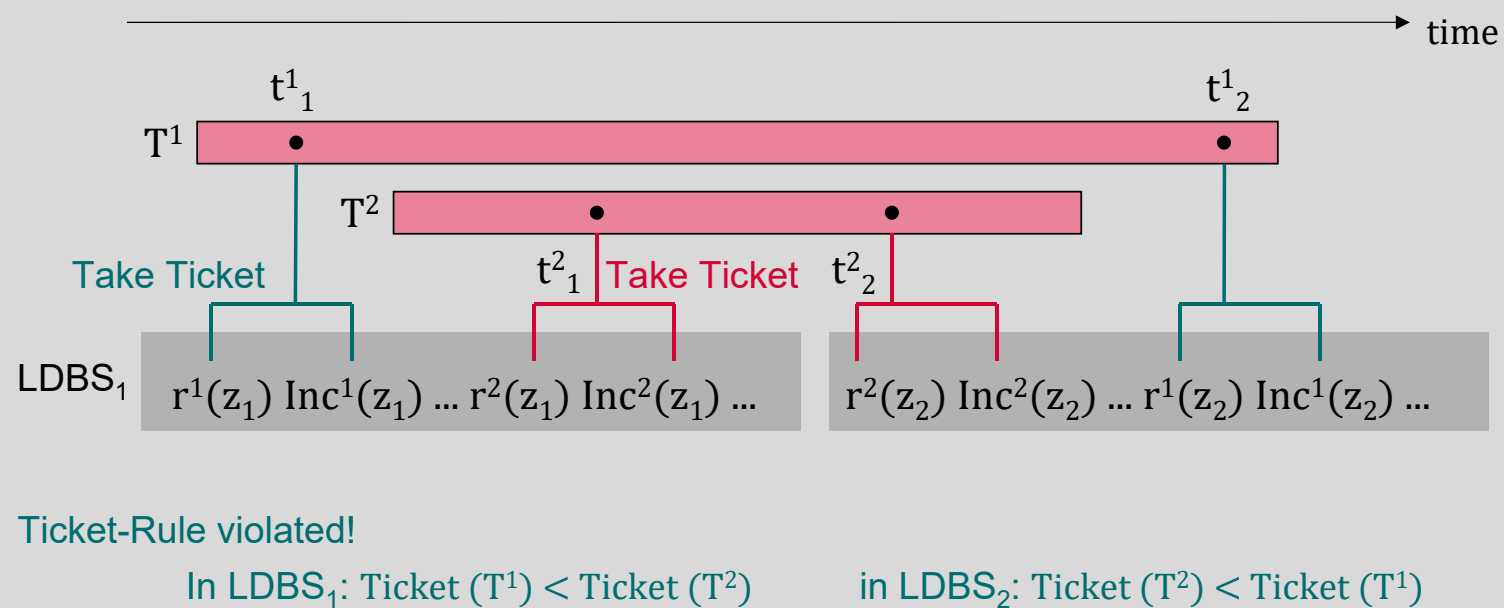
⇒ Prerequisite for correctness in federated DBMS is also the application of S2PL in each component database system.

- In the previous example, S2PL has enforced that T^L is completely executed before T^1 . Then, it would have not been possible for the local transaction T^L to completely encompass both sub-transactions and they would have not been (transitively) in conflict.

Influence of Heterogeneity: Tickets ...

- A core characteristics of federated systems is the heterogeneity of component databases. Therefore, nobody can rely on the S2PL support of all component databases (Oracle, one of the market leaders in the field, for instance, does not; the same holds for the widely used open source databases PostgreSQL and MySQL).
- With the deviation **via artificial increments (writes) on an additional data object (“Ticket”)** in each local database system, correctness can be achieved even without S2PL.
 - This requires that **each sub-transaction first “Takes a Ticket”** which means reading the current value of a counter and subsequently incrementing it.
 - Globally, it has to be guaranteed that the tickets are in the same order in all component databases. **This forced local conflict of all transactions finally leads to a identical serialization order.**

... Influence of Heterogeneity: Tickets



Timestamp Ordering (TO) ...

- Assumption:
 - A **unique timestamp** $ts(T^k)$ is explicitly assigned to each transaction T^k
 - Usually, this is BOT
 - In the ticketing approach, this assignment was done implicitly at the time the ticket was taken
- All timestamps $ts(T^k)$ have to be element of a strongly monotonically increasing series. This means, that clocks have to be synchronized to make sure that timestamps of distributed transactions have the same order on all sites.
- **TO Rule:** for each pair of conflicting actions $p^i(x)$ and $q^k(x)$ with $i \neq k$, the following has to hold:
$$p^i(x) < q^k(x) \Leftrightarrow ts(T^i) < ts(T^k)$$

... Timestamp Ordering (TO) ...

- For each database object, the following information is maintained:
 - $\text{max_w_ts}(x)$ highest timestamp of a transaction that has written x
 - $\text{max_r_ts}(x)$ highest timestamp of a transaction that has read x
- For each active transaction T^k , the following data is managed:
 - $\text{Commit}(T^k)$ **false** when T^k starts
true after commit of T^k
 - $\text{WS}(T^k)$ Write set of T^k , i.e., the set of all objects T^k has written so far

... Timestamp Ordering (TO)

Protocol of a TO scheduler:

- T^k wants to read x :
 - if $ts(T^k) < \max_w_ts(x)$: T^k requests a value that is already outdated (has already been overwritten by a younger transaction) $\Rightarrow T^k$ has to be aborted
 - Otherwise, if $ts(T^k) \geq \max_w_ts(x)$: read request is granted. Update $\max_r_ts(x)$ if $ts(T^k) > \max_r_ts(x)$.
- T^k wants to write x :
 - if $ts(T^k) < \max_r_ts(x)$: there is a younger transaction that has read x already. Write request cannot be granted $\Rightarrow T^k$ has to be aborted
 - if $ts(T^k) < \max_w_ts(x)$: write is unnecessary (x has already been overwritten)
 - Otherwise, if $ts(T^k) \geq \max_r_ts(x) \wedge ts(T^k) \geq \max_w_ts(x)$: write request can be granted, $\max_w_ts(x)$ has to be updated.

Strict Timestamp Ordering (STO)

- The TO protocol does not only allow a transaction T^k to read from an active transaction T^i , it also allows T^k to commit before T^i
- Strict timestamp ordering (STO) therefore prevents reads from active transactions
- The write rules of STO are the same as in TO.
- However, the read rule has to be extended: T^k is allowed to read x :
 - if $ts(T^k) \geq \max_w_ts(x)$ – if the requested value has not been overwritten (as before), and
 - If $x \in WS(T^i)$, then $Commit(T^i) = \text{true}$ has to hold. Otherwise, T^k has to wait (since T^i could be aborted, and in this case the undo of the write of T^i – which is again a write – would violate the TO rule)

Summary: Correctness of Distributed Transactions

- Imposing a serialization order in each component database system does unfortunately not solve all problems of distributed transactions. It just ensures that everything is synchronized correctly in case no failure occurs. In addition, also global atomicity needs to be provided. Thus, the following two requirements for distributed transactions can be derived):
 1. Requirement: a distributed protocol that enforces an atomic commit over all participating component databases: **2PC (Two Phase Commit)**
 2. Requirement: local protocols that guarantee commit order preserving serializability (serialization order then coincides with the commit order):
S2PL (Strict 2 Phase Locking) – or protocols like the **ticketing protocol** or **strict timestamp ordering** that report the inner serialization order of a DBMS to the outside.

⇒ Only **S2PL** and **2PC** are needed for correctly supporting distributed transactions!

Recap: Questions

- What are the particular challenges when enforcing atomicity in a distributed database compared to a centralized database?
- What are the differences (and commonalities) between 2PC and 3PC?
- What are the different roles in the X/Open protocol?
- What are the differences (and commonalities) between multi database transactions and federated transactions?
- Which protocols can enforce commit order preserving serializable transaction executions?
- What alternative to lock-based protocol for correct concurrency control exist?

Further Reading

- [WV 02] G. Weikum, G. Vossen: *Transactional Information Systems*, Morgan Kaufmann, 2001.
[General Overview]
- [GR 93] J. Gray, A. Reuter: *Transaction Processing: Concepts and Techniques*, Morgan Kaufman, 1993.
[Systems-oriented Overview]
- [BN 97] P. Bernstein, E. Newcomer: *Principles of Transaction Processing*, Morgan Kaufman Publishers, 1997.
[Basic Transaction Concepts]